

BITS PILANI

M.Tech in Data Science & Artificial Intelligence

Artificial Neural Networks

Condensed Revision Notes

Complete Course Cram Sheet — Modules 1 to 13

Course	M.Tech Data Science & AI, BITS Pilani
Subject	Artificial Neural Networks (ANN)
Scope	Weeks 1–13, all modules
Format	Definitions, formulas, tables, checklists
Date	01 August 2026

Table of Contents

1 Foundations: AI, ML, DL and the Artificial Neuron . . . 2	8.7 Evolution of CNN Architectures 27
1.1 AI vs. ML vs. DL 2	9 Sequence Models: RNN, LSTM, GRU, Attention 28
1.2 Why Deep Learning Works Today — the 4 Pillars 2	9.1 Why We Need Sequence Models 28
1.3 The Artificial Neuron 2	9.2 Recurrent Neural Networks (RNNs) 28
1.4 Weights, Bias, Layers 2	9.3 BPTT and the Vanishing Gradient in RNNs 28
1.5 Feed-Forward Neural Network (FFN) 3	9.4 Limitations of RNNs Beyond Vanishing Gradients 28
1.6 Where Neural Networks Are Used 3	9.5 Long Short-Term Memory (LSTM) 29
1.7 Limitations of Shallow Networks → Motivation for Depth . . 3	9.6 Gated Recurrent Units (GRU) 29
2 Mathematical Foundations 5	9.7 Attention 30
2.1 Vectors, Matrices, Tensors 5	9.8 Transformers 30
2.2 Shape Consistency and Matrix Multiplication 5	10 Model Evaluation and Performance Metrics 32
2.3 Dot Product 5	10.1 Why Accuracy Alone Is Not Enough 32
2.4 Layer Computation with Matrices 5	10.2 Confusion Matrix and Classical Metrics 32
2.5 Derivatives, Partial Derivatives, Gradients 5	10.3 Loss Surfaces and Optimisation Geometry 32
2.6 The Chain Rule 6	10.4 Flat vs. Sharp Minima 33
2.7 Numerical Stability 6	10.5 Overfitting Patterns in Deep Models 33
3 Perceptron and Logistic Neuron 7	10.6 Calibration 33
3.1 The Perceptron 7	10.7 Improving Calibration 34
3.2 Geometric Interpretation 7	10.8 Robustness 34
3.3 The Perceptron Learning Rule 7	10.9 Uncertainty and Softmax Entropy 34
3.4 Linear Separability and XOR 7	11 Model Diagnostics and Debugging 36
3.5 Multi-Layer Networks Solve XOR 8	11.1 From Evaluation to Diagnostics 36
3.6 The Logistic Neuron 8	11.2 Gradient Flow Diagnostics 36
3.7 Perceptron vs. Logistic Neuron 8	11.3 Activation Diagnostics 36
4 Multi-Layer Perceptrons and Activation Functions . 10	11.4 Parameter Diagnostics 37
4.1 What is an MLP? 10	11.5 BatchNorm Diagnostics 38
4.2 Layer Parameters and the Forward Pass 10	11.6 A Structured Debugging Workflow 38
4.3 Hidden Layers and Representation Learning 10	11.7 Monitoring Tools and Experiment Tracking 38
4.4 Expressive Power: Depth vs. Width 10	12 Hyperparameter Tuning and Experiment Control . 40
4.5 Why We Need Activation Functions 11	12.1 Why Manual Tuning Fails 40
4.6 Sigmoid and Tanh 11	12.2 The Hyperparameters That Matter 40
4.7 The ReLU Family 11	12.3 Learning Rate — Symptoms of a Bad Choice 40
4.8 Softmax for Multi-Class Outputs 12	12.4 Batch Size 40
4.9 Choosing Activations — Practical Guidelines 12	12.5 Automated Hyperparameter Search 41
5 Backpropagation and Training Dynamics 13	12.6 Bayesian Optimisation 41
5.1 The Training Pipeline 13	12.7 A Systematic Experimentation Workflow 41
5.2 What a Gradient Means 13	12.8 Reproducibility 42
5.3 Computational Graphs 13	13 Explainability, Responsible AI and Future Trends . 43
5.4 Backpropagation Step by Step (Two-Layer Network) 13	13.1 Why Explainability Matters 43
5.5 Loss Functions 14	13.2 Interpretability vs. Explainability 43
5.6 Vanishing and Exploding Gradients (Canonical Treatment) 14	13.3 Explainability Methods 43
5.7 Weight and Bias Initialization 15	13.4 Bias and Fairness 44
5.8 Mitigating Vanishing / Exploding Gradients 15	13.5 Future Directions 45
6 Optimisation Algorithms 17	14 Quick Reference Sheet 47
6.1 What is Optimisation? 17	14.1 Activation Functions at a Glance 47
6.2 Gradient Descent 17	14.2 Optimizers at a Glance 47
6.3 Variants of Gradient Descent 17	14.3 Regularisation Techniques at a Glance 47
6.4 Momentum 17	14.4 Hyperparameter Cheat Sheet — Too High vs. Too Low . . 48
6.5 Why Adaptive Optimizers Were Needed 17	14.5 Symptom → Diagnosis → Fix 48
6.6 RMSProp 18	14.6 Formula Sheet 48
6.7 Adam (Adaptive Moment Estimation) 18	14.7 Final Exam-Day Checklist 49
6.8 Learning Rate Scheduling 19	
6.9 Gradient Clipping (Full Treatment) 19	
6.10 Common Optimisation Pitfalls 19	
7 Regularisation and Generalisation 21	
7.1 Generalisation 21	
7.2 Underfitting, Overfitting, Capacity 21	
7.3 Bias-Variance Trade-off 21	
7.4 Detecting Overfitting 21	
7.5 L1 and L2 Regularisation 21	
7.6 Dropout 22	
7.7 Batch Normalisation 22	
7.8 Early Stopping 22	
7.9 Data Augmentation 23	
7.10 Implicit Regularisation from the Training Setup 23	
7.11 Combining Regularisation Methods 23	
7.12 Practical Checklist for Preventing Overfitting 23	
8 Convolutional Neural Networks 25	
8.1 Why Fully Connected Networks Fail on Images 25	
8.2 The Convolution Operation 25	
8.3 Filters and Feature Maps 25	
8.4 Channels and Multiple Filters 25	
8.5 Stride, Padding, Pooling 26	
8.6 The Standard CNN Block and Full Pipeline 26	

1 Foundations: AI, ML, DL and the Artificial Neuron

1.1 AI vs. ML vs. DL

The Nested Relationship: AI $\supset$ ML $\supset$ DL

- **AI** — any goal-driven system that perceives, decides, and acts. Even if-then rules count.
- **ML** — data-driven AI. Learns a function $f(x; \theta)$ from data instead of being given rules.
- **DL** — ML using deep neural networks. Features **and** parameters are learned jointly.

Every ML system has 4 components: data, model, loss function, optimisation method.

Feature engineering bottleneck — in classical ML humans hand-design features (edges, TF-IDF, MFCC) and the model only learns weights on top. Performance is capped by feature quality, not the algorithm. Does not scale to high-dimensional data.

Aspect	Classical ML	Deep Learning
Features	Hand-designed by humans	Learned from data
What is learned	Parameters only	Features + representations + parameters
Structure	One fixed function	Composition of many layers
Scales to complex data?	Poorly	Very well
Best data type	Structured / tabular	Unstructured (image, audio, text)

1.2 Why Deep Learning Works Today — the 4 Pillars

All four are needed **simultaneously**; no single one is sufficient.

Pillar	Why it matters
Large-scale data	Millions of images / billions of words; reduces estimation error, stabilises very large models.
Massive compute	GPUs/TPUs parallelise the matrix multiplications that dominate training.
Robust optimisation	ReLU (better gradient flow), Xavier/He init (stable signal), Adam/RMSProp (faster convergence) — together solved vanishing/exploding gradients.
Powerful architectures	CNN (spatial locality), RNN (temporal structure), Transformer (long-range attention) act as structural priors → far more sample-efficient.

1.3 The Artificial Neuron

Biological neuron	Artificial neuron
Dendrites (receive signals)	Input features x_i
Soma (integrates signals)	Weighted sum $z = \sum w_i x_i + b$
Firing / action potential	Activation function $a = \varphi(z)$

Core Equation of an Artificial Neuron

$$z = \sum_{i=1}^n w_i x_i + b, \quad a = \varphi(z)$$

x_i = inputs, w_i = weights (learned), b = bias (learned), φ = non-linear activation.

- A neuron does **one** thing: weighted evidence aggregation, then a non-linearity.
- The weight vector acts like a **template**; strong response when input matches the template.
- **Key limitation:** a single neuron detects only one simple linear pattern → one simple decision boundary, very limited expressive power.

1.4 Weights, Bias, Layers

- **Weight** — importance/influence of one input. Larger $|w_i|$ → matters more; negative w_i → input reduces output.
- **Bias** — baseline output; shifts **when** the neuron starts to activate. Lets a neuron respond even when inputs are zero.

- **Layer** — a group of neurons at the same depth.

Layer type	Role
Input	Holds raw features. No computation.
Hidden	Computes intermediate/abstract representations. The real transformation.
Output	Produces the final task-specific prediction.

1.5 Feed-Forward Neural Network (FFN)

Definition

A feed-forward network forms a **Directed Acyclic Graph (DAG)**: information flows one way only (input → hidden → output). No loops, no feedback, no dependence on past outputs. Each layer depends only on the previous layer's output.

- Simplifies forward computation (no circular dependencies).
- Enables efficient learning via backpropagation.
- Stacking simple layers represents highly complex non-linear functions.

Universal Approximation Theorem (intuition) — a feed-forward network with enough neurons/ layers can approximate any continuous function mapping inputs to outputs. (*full statement in W4*)

1.6 Where Neural Networks Are Used

- **Computer vision** — pixels → labels / boxes / segmentation. Early layers: edges; deep layers: objects.
- **Speech & NLP** — speech-to-text, translation, chatbots, LLMs (RNNs, Transformers).
- **Recommenders** — learned user/item representations predict engagement.
- **Finance, healthcare, robotics, manufacturing** — fraud detection, medical imaging, navigation, predictive maintenance.

When to use a neural network: unstructured data (images, audio, raw text), very large datasets, and problems needing end-to-end representation learning.

When NOT to: small structured/tabular data — XGBoost / Random Forest / logistic regression are often as good, and are cheaper, faster, lower-latency and more interpretable. Deep models cost compute, add latency, and are black-box.

1.7 Limitations of Shallow Networks → Motivation for Depth

Limitation	Explanation
Limited representational power	Few transformations → cannot capture complex non-linear relations or variable interactions.
Feature engineering bottleneck	Performance capped by hand-designed feature quality; does not scale.
No hierarchical structure	Treats data “flatly”, but real data is hierarchical (pixels→edges→shapes→objects).

Wide network	Deep network
More neurons in one layer	More layers stacked
Many features at one abstraction level	Features built gradually across levels
Can need exponentially many neurons	Usually far fewer parameters
No match to data hierarchy	Natural fit for vision/language

Rule: no fixed formula for right depth/width — decided experimentally. Start simple and small (avoids high variance / overfitting), increase depth only if needed.

MUST REMEMBER

- AI $\supset$ ML $\supset$ DL. DL learns **features and parameters jointly**; classical ML only learns parameters on hand-made features (the feature-engineering bottleneck).
- 4 pillars of modern DL, all needed together: **data, compute, optimisation, architecture**.
- Every ML system has 4 components: **data, model, loss, optimiser**.
- A neuron: $z = \sum w_i x_i + b$, then $a = \varphi(z)$ — weighted aggregation plus a non-linearity.
- A single neuron = one simple linear pattern detector, very limited expressive power.
- Weights = feature importance; bias = baseline / activation-threshold shift; layers = hierarchical abstraction.
- Feed-forward network = one-directional flow, a **DAG**, no loops or feedback.
- Shallow nets fail on complex data: limited representational power, feature-engineering bottleneck, no hierarchy.
- Depth beats width for complex tasks because deeper layers **reuse and compose** what earlier layers learned.

2 Mathematical Foundations

2.1 Vectors, Matrices, Tensors

- **Vector** — ordered list of numbers = 1st-order tensor. n elements $\rightarrow$ shape $(n,)$.
- **Matrix** — 2D grid of numbers = 2nd-order tensor. Shape $(m \times n)$ = rows $\times$ columns.
- **Tensor** — general multi-dimensional array. 3+ dimensions (e.g. an RGB image: height $\times$ width $\times$ channels).

In a network: one sample = vector; layer weights = matrix; RGB image = tensor. PyTorch/TensorFlow store everything as one “tensor” type.

2.2 Shape Consistency and Matrix Multiplication

The Matrix Multiplication Rule

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times p} \implies AB \in \mathbb{R}^{m \times p}$$

Inner dimensions must match. Output takes the two **outer** dimensions. If inner dimensions differ, multiplication is invalid and fails in code.

Exam note: shape mismatches are the single most common neural-network implementation bug.

2.3 Dot Product

Dot Product — Algebraic and Geometric

$$a \cdot b = \sum_{i=1}^n a_i b_i \quad a \cdot b = \|a\| \quad \|b\| \cos \theta$$

Angle θ	Dot product	Meaning
0° (same direction)	Large positive	Strongly aligned / similar
180° (opposite)	Large negative	Opposed
90° (perpendicular)	Exactly 0	No similarity

This is the idea behind **cosine similarity**. Every neuron computes a dot product between the input and its weight vector: it measures **how similar the input is to the pattern the neuron seeks**; the bias shifts where activation begins.

2.4 Layer Computation with Matrices

Full Layer Forward Computation

$$z = Wx + b$$

- $W \in \mathbb{R}^{m \times n}$: m = neurons in layer, n = input features (one row per neuron).
- $x \in \mathbb{R}^n$: input vector. $b \in \mathbb{R}^m$: one bias per neuron. $z \in \mathbb{R}^m$: one raw output per neuron.

For a **batch**, stack input vectors as columns of X ; one multiplication WX computes the whole batch at once. This parallelism is exactly why GPUs accelerate deep learning.

2.5 Derivatives, Partial Derivatives, Gradients

- **Derivative** — rate of change / sensitivity — slope of a function at a point.

$$\frac{dy}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- **Partial derivative** — sensitivity of the output to **one** variable while all others are held fixed. “Turn one knob at a time.”
- **Gradient vector** — collects every partial derivative into one vector.

Gradient Vector

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

Same dimension as the input — one sensitivity value per input.

Geometric meaning:

- Points in the direction of **steepest increase**.
- Magnitude = how fast the function changes (large = steep, near zero = locally flat).
- Gradient = 0 → stationary point (minimum, maximum, or saddle).
- Always perpendicular to level curves.

Learning uses — $-\nabla f$, the direction of steepest **decrease**, because we minimise loss.

2.6 The Chain Rule

Chain Rule

If y depends on x only through an intermediate u , i.e. $u = g(x)$ and $y = f(u)$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Total sensitivity = **product of local sensitivities** along the chain.

In a deep network, the loss reaches an early-layer weight only through every layer in between, so its gradient is a product of local derivatives across all of them. Repeated application of this rule, layer by layer, **is** backpropagation. (*full algorithm in W5*)

2.7 Numerical Stability

Problem	Description
Overflow	Number too large to represent → stored as ∞ . Every later computation using it becomes meaningless.
Underflow	Number too small → rounded to exactly 0. Gradient/probability information is permanently lost.

Deep learning is prone to instability because of: (1) long chains of multiplications, (2) heavy use of exponentials, (3) extremely small probabilities.

Log-Sum-Exp Trick

$$\log \sum_i e^{x_i} = c + \log \sum_i e^{x_i - c}, \quad c = \max_i x_i$$

After subtracting the max, the largest term becomes $e^0 = 1$ and all others lie in $(0, 1]$ — nothing overflows, nothing vanishes. Applied automatically inside softmax and cross-entropy.

Other stabilisation techniques: work in log domain; subtract the max before exponentiating; add a small ϵ to denominators; replace long products of small probabilities with log-sums.

MUST REMEMBER

- Vector = 1st-order tensor; matrix = 2nd-order; tensor = general multi-dimensional array.
- Matrix multiplication needs **matching inner dimensions**; output takes the outer two. Shape mismatch is the 1 implementation bug.
- Dot product $a \cdot b = \sum a_i b_i = \|a\| \|b\| \cos \theta$ — positive if aligned, negative if opposite, **exactly zero if perpendicular**.
- A neuron computes $w^T x + b$; a full layer computes $Wx + b$ for all neurons at once; batches stack inputs for parallel GPU computation.
- Derivative = sensitivity; partial derivative = sensitivity to one variable with the rest fixed.
- Gradient ∇f = vector of all partials; points toward **steepest increase**; $-\nabla f$ drives learning; zero gradient = stationary point.
- Chain rule $\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$ — required whenever a variable affects the output only through an intermediate.
- **Backbone of backpropagation.**
- Overflow → ∞ ; underflow → 0. The **log-sum-exp trick** (subtract the max before exponentiating) keeps softmax/cross-entropy safe.

3 Perceptron and Logistic Neuron

3.1 The Perceptron

The simplest computational neuron model (Rosenblatt, late 1950s); the first true neural network model. Job: **binary classification**.

Perceptron Equation

$$z = w^T x + b, \quad \hat{y} = \text{sign}(z) = \begin{cases} +1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$$

Two steps: (1) compute a linear score z ; (2) hard-threshold it into a class. Because step 1 is linear, $w^T x + b = 0$ is a **line** in 2D, a **plane** in 3D, a **hyperplane** in higher dimensions.

3.2 Geometric Interpretation

Element	Geometric role
Weight vector w	Perpendicular (normal) to the decision boundary. Sets its orientation (rotation). Points toward the +1 region.
Bias b	Sets the position . Shifts the boundary parallel to itself, without rotating it.

- 2D boundary: $w_1 x_1 + w_2 x_2 + b = 0$.
- Classification is a pure **sign test**: which side of the line does the point fall on?
- No distance, no probability, no confidence — only side-of-line membership.

3.3 The Perceptron Learning Rule

Mistake Condition and Update

Mistake occurs when

$$y_i(w^T x_i + b) \leq 0$$

Then, and **only** then, update:

$$w \leftarrow w + \eta y_i x_i, \quad b \leftarrow b + \eta y_i$$

η = learning rate (step size).

Why it works: a wrongly-negative positive example adds a positive multiple of x to w , increasing alignment; a wrongly-positive negative example subtracts it, reducing alignment. Every update nudges the boundary to fix the mistake just seen.

Properties:

- **Mistake-driven**, not continuously optimising on every example.
- Starts from random weights; stops when zero mistakes on the training set.
- Does **not** minimise a smooth loss, does **not** use gradients, does **not** give probabilities.

Perceptron Convergence Theorem

If the training data is **linearly separable**, the algorithm is guaranteed to converge in a **finite** number of updates. If the data is **not** linearly separable, it **never** converges — weights keep updating indefinitely.

3.4 Linear Separability and XOR

Linear separability — a data set is linearly separable if $\exists w, b$ such that $y_i(w^T x_i + b) > 0$ for every training example — all positives on one side of some hyperplane, all negatives on the other.

Whether a perceptron can succeed is a **geometric property of the data**, not of how well it is trained.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1

The XOR problem. Output is 1 when inputs differ, 0 when they are the same. Plotted in the $x_1 x_2$ plane, the two class-1 points sit **diagonally opposite** each other, as do the two class-0 points. No single straight line can separate diagonal pairs. XOR is the

x_1	x_2	XOR
1	1	0

clearest demonstration of the single-layer perceptron's fundamental limit.

3.5 Multi-Layer Networks Solve XOR

The Key Idea

A hidden layer creates **multiple** linear splits instead of one. Each hidden neuron learns its own linear separator; the output neuron **recombines** them into a final decision. Composing multiple linear cuts forms **non-linear decision regions**, even though each individual neuron is still linear.

For XOR: two hidden neurons create two lines; points **between** the lines get one class, points outside get the other. Requires at least one hidden layer. Multilayer networks are **strictly more expressive** than single perceptrons — this jump in expressive power (not a better learning rule) is why deeper networks beat shallow ones.

3.6 The Logistic Neuron

The perceptron's hard threshold gives a label but never a **confidence**: no uncertainty, no probabilistic interpretation, no smooth optimisation. The logistic neuron replaces the step with a smooth sigmoid.

Sigmoid (Logistic) Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{y} = \sigma(w^T x + b)$$

Key values: $\sigma(0) = 0.5$, $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$, $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$. Range $(0, 1)$.

Probabilistic interpretation: $\hat{y} = P(y = 1 | x)$. Near 1 $\rightarrow$ confident class 1; near 0 $\rightarrow$ confident class 0; near 0.5 $\rightarrow$ uncertain.

Crucially, the decision boundary stays linear: $\hat{y} = 0.5 \iff w^T x + b = 0$ — the same hyperplane as the perceptron. What changes is not **where** the boundary is, but **how sharply** the output transitions across it.

3.7 Perceptron vs. Logistic Neuron

Aspect	Perceptron	Logistic Neuron
Activation	Hard step / sign	Smooth sigmoid
Output range	Discrete $\{-1, +1\}$	Continuous $(0, 1)$
Interpretation	Class label only	Probability $P(y = 1 x)$ + confidence
Transition at boundary	Abrupt jump	Smooth, gradual
Differentiable?	No	Yes — enables gradient-based training
Decision boundary	$w^T x + b = 0$ (linear)	$w^T x + b = 0$ (linear — identical)
Near the boundary	Confidently ± 1 even for borderline points	Output near 0.5, explicitly shows uncertainty

Why logistic neurons win in practice: differentiability enables gradient-based training; probabilistic outputs let us rank by confidence, tune decision thresholds per application, and reason about uncertainty. This makes the logistic neuron the building block of modern multilayer networks.

MUST REMEMBER

- Perceptron: $z = w^T x + b$, $\hat{y} = \text{sign}(z)$ — a hard-threshold **linear** classifier.
- Weight vector w sets the boundary's **orientation**; bias b sets its **position**.
- Learning rule is **mistake-driven**: update only when $y_i(w^T x_i + b) \leq 0$.
- Convergence theorem: guaranteed convergence **only if** the data is linearly separable; otherwise never converges.
- XOR is the classic non-linearly-separable problem — the two classes sit diagonally opposite, so no single line works.
- A hidden layer creates **multiple** linear splits which recombine into non-linear regions — this solves XOR and makes multilayer nets strictly more expressive.
- Logistic neuron: same z , but $\hat{y} = \sigma(z)$, read as $P(y = 1|x)$.
- **The decision boundary is identical (linear) for both** — only the smoothness of the output transition differs.
- Logistic neurons win because they are **differentiable** (gradient training) and give **confidence**, not just a label.

4 Multi-Layer Perceptrons and Activation Functions

4.1 What is an MLP?

MLP — a feed-forward, **fully connected (dense)** neural network with one or more hidden layers. Every neuron in a layer connects to every neuron in the next.

- **Input layer** — holds the raw feature vector. No computation.
- **Hidden layer(s)** — where the actual transformation happens. Called “hidden” only because we do not directly observe their values.
- **Output layer** — converts the last hidden representation into the final prediction.

MLPs are the conceptual foundation of deep learning; CNNs, RNNs and Transformers reuse the same layered-computation + non-linearity ideas.

4.2 Layer Parameters and the Forward Pass

Layer Equation

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \varphi^{(l)}(z^{(l)})$$

An MLP is a **composition of simple functions**, one per layer, not one giant formula.

4 Steps Inside One Neuron (the Forward Pass)

1. Multiply each input by its corresponding weight.
2. Sum all weighted inputs.
3. Add the bias term.
4. Apply the activation function.

No learning happens during a forward pass — it is pure computation. Each layer sees only the **transformed** representation from the previous layer, never the original input again.

Parameter Count

A fully connected layer from n inputs to m neurons has $n \times m$ weights + m biases. More connections $\rightarrow$ more capacity, but also more compute and higher overfitting risk.

Worked shapes (2 features $\rightarrow$ 3 hidden neurons $\rightarrow$ 1 output):

Quantity	Shape	Meaning
x	(2,)	2 raw features
$W^{(1)}$	(3 $\times$ 2)	3 neurons, each sees 2 inputs
$b^{(1)}$	(3,)	one bias per neuron
$a^{(1)}$	(3,)	output of hidden layer 1
$W^{(2)}$	(1 $\times$ 3)	1 output neuron, sees 3 hidden values
$\hat{y}$	(1,)	final prediction

4.3 Hidden Layers and Representation Learning

- **Representation** — how the data is encoded inside the network. Raw input is the first representation; each hidden layer produces a new transformed version.
- A hidden neuron computes $\varphi(w^T x + b) \rightarrow$ it behaves as a **feature detector**, responding strongly to a specific pattern. A layer is a **bank** of feature detectors.
- **Hierarchical abstraction**: pixels $\rightarrow$ edges $\rightarrow$ corners/contours $\rightarrow$ shapes/object parts $\rightarrow$ object identity.
- Width = more feature detectors at one level. Depth = more levels of abstraction. Both raise capacity, compute, and overfitting risk.

4.4 Expressive Power: Depth vs. Width

Universal Approximation Theorem (UAT)

A feed-forward network with **even one hidden layer** can approximate any continuous function on a compact domain, provided it has **enough neurons**. **The catch:** a wide-but-shallow network may need an exponentially large number of neurons. Deep networks represent the same functions with far fewer total parameters, because depth allows re-use and composition of simpler features.

Real data is hierarchical (edges→shapes→objects; characters→words→meaning), so depth matches the structure of the problem — this is why modern design favours depth over extreme width.

4.5 Why We Need Activation Functions

The Linear Collapse Problem

With purely linear layers, $z^{(2)} = W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)} = W'x + b'$. **No matter how many linear layers are stacked, the whole network collapses to one linear model.** It can only learn straight-line boundaries and can never solve XOR.

Activation functions break the collapse: each layer becomes a genuinely non-linear transformation of the previous one, so layer 1 learns non-linear features of the input, layer 2 learns non-linear combinations of those, and so on. **Activation functions are the true source of expressive power.**

4.6 Sigmoid and Tanh

Sigmoid and Tanh

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{range } (0, 1), \quad \sigma(0) = 0.5$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \text{range } (-1, 1), \quad \tanh(0) = 0$$

- Sigmoid: S-shaped, output readable as a probability → standard for the **binary classification output layer**.
- Tanh: rescaled/shifted sigmoid, **zero-centered**; historically preferred over sigmoid in hidden layers because balanced positive/negative activations optimise better.
- Different layers of the same network may use different activations.

Saturation — for very large $|z|$ the curve flattens, so a big change in input produces almost no change in output. Gradients there become tiny → a main contributor to **vanishing gradients**. This is also why we normalise/standardise inputs — to keep values out of the flat region.

4.7 The ReLU Family

ReLU (Rectified Linear Unit)

$$\text{ReLU}(z) = \max(0, z)$$

Derivative is exactly 1 for $z > 0$ and 0 for $z < 0$ — piecewise-linear and very cheap.

Why ReLU is the default hidden activation:

- Does not saturate for positive inputs → information flows through many layers.
- Much faster, more stable training than sigmoid/tanh.
- Produces **sparse** activations (many exact zeros) → often better generalisation and efficiency.
- Extremely cheap to evaluate.

Dying ReLU — if training pushes a neuron so that $z < 0$ for (almost) every input, it outputs 0 always. Since the gradient is also 0 there, it receives **no** gradient signal and can **never recover** — a permanently dead neuron that reduces effective capacity.

Variants that Fix Dying ReLU

$$\text{LeakyReLU}(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases}, \quad \alpha \approx 0.01 \text{ (fixed)}$$

PReLU: same form, but α is a **learnable** parameter tuned during training. More flexible, a few extra parameters; used in some high-performance vision models.

4.8 Softmax for Multi-Class Outputs

Logits — raw, unnormalised real-valued class scores from the last linear layer (can be any sign or size).

Softmax Definition

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Every output lies in $(0, 1)$; all outputs sum to exactly 1; larger logits get larger probabilities.

Worked example. Logits $[2, 1, 0.1] \rightarrow$ exponentials $\approx [7.4, 2.7, 1.1]$, sum $\approx 11.2 \rightarrow$ probabilities $\approx [0.66, 0.24, 0.10]$. Predicted class = highest probability (Class 1).

	Sigmoid	Softmax
Output	One probability	A vector of probabilities, one per class
Constraint	Value in $(0, 1)$	All values sum to exactly 1 (classes compete)
Used for	Binary classification	Mutually-exclusive multi-class classification
Location	Output layer only	Output layer only, never a hidden layer

4.9 Choosing Activations — Practical Guidelines

Problem	Cause and effect
Saturation	Sigmoid/tanh flatten for large $ z \rightarrow$ very weak learning signals.
Vanishing gradients	Saturation compounds backward through layers; small values multiply and shrink $\rightarrow$ early layers barely learn.
Dying ReLU	Neurons permanently output 0 $\rightarrow$ reduced effective capacity.
Exploding activations	Activations grow huge from poor init, excess depth, or unstable weight interaction $\rightarrow$ numerical overflow, training failure.

Hidden layers: use ReLU by default $\rightarrow$ if many neurons appear dead, try Leaky ReLU $\rightarrow$ for very deep networks always prefer a ReLU-family activation $\rightarrow$ avoid sigmoid in deep hidden layers.

Output Layer Activation by Task

Task	Output neurons	Activation
Binary classification	1	Sigmoid — one probability
Multi-class classification	K	Softmax — probability vector
Regression	1 per predicted value	Linear (no activation)

Chosen for **correct interpretation** of predictions, not for training speed.

MUST REMEMBER

- MLP = input + hidden layer(s) + output, feed-forward, fully connected. Each layer: $z = Wa + b$, then $a = \varphi(z)$.
- Forward pass = 4 steps per neuron (multiply, sum, add bias, activate). **No learning happens during it.**
- Hidden layers do **representation learning** — each neuron is a feature detector; raw $\rightarrow$ derived $\rightarrow$ hierarchical features.
- Width = more detectors per level; depth = more levels. Depth is usually far more parameter-efficient (UAT caveat: one hidden layer suffices **in theory**, but may need exponentially many neurons).
- Without activations, stacked linear layers collapse into one linear model, no matter how deep.**
- Sigmoid $(0, 1)$ for binary output; tanh $(-1, 1)$ zero-centered; **both saturate $\rightarrow$ vanishing gradients.**
- ReLU = $\max(0, z)$ is the default hidden activation — fast, sparse, non-saturating for $z > 0$; risk = **dying ReLU**; Leaky ReLU / PReLU fix it with a small negative slope.
- Softmax turns logits into a probability distribution over K mutually exclusive classes — **output layer only.**
- Output activation is chosen **by task**: sigmoid (binary), softmax (multi-class), linear (regression).

5 Backpropagation and Training Dynamics

5.1 The Training Pipeline

The 4 Steps of Training

1. **Forward pass** — compute prediction from current weights and biases.
2. **Loss computation** — compare prediction with truth to get an error number.
3. **Gradient computation** — find how much each parameter contributed to that error. **This step is backpropagation.**
4. **Parameter update** — nudge each parameter to reduce the error. **This is optimisation, not backprop.**

Critical distinction: backpropagation only **computes gradients**. It does not choose a step size and does not move the weights — that is the optimiser’s job, using a learning rate. (W6)

5.2 What a Gradient Means

Gradient	Interpretation
Positive	Increasing the parameter increases the error → decrease it.
Negative	Increasing the parameter decreases the error → increase it.
Near zero	This parameter has almost no local effect on the error.
Large magnitude	Strong influence → a big correction is justified.

Sign = **direction**; magnitude = **strength**. A gradient is a per-parameter “responsibility report”.

5.3 Computational Graphs

Computational graph — a picture of a mathematical expression: nodes are variables/intermediate values, edges are operations. Shows exactly how the output is built from the input.

- **Local gradient** — derivative of a node’s output w.r.t. its **direct** input only.
- **Global gradient** = **product of local gradients** along the path from input to output (chain rule).
- The **same graph is reused for both passes**: forward pass evaluates and **caches** every intermediate value; backward pass walks backward multiplying local gradients using the cached values — no recomputation.
- Backpropagation is simply the systematic backward traversal of this graph.

5.4 Backpropagation Step by Step (Two-Layer Network)

Forward: $z_1 = w_1x + b_1$, $a_1 = \varphi(z_1)$, $z_2 = w_2a_1 + b_2$, $\hat{y} = z_2$, $L = \frac{1}{2}(\hat{y} - y)^2$. Cache $x, z_1, a_1, z_2, \hat{y}$.

Backward Pass — 5 Steps

1. **Start at the output:**

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

2. **Output-layer parameters:**

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot a_1, \quad \frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial \hat{y}}$$

3. **Propagate into the hidden activation:**

$$\frac{\partial L}{\partial a_1} = \frac{\partial L}{\partial \hat{y}} \cdot w_2$$

4. **Pass through the activation:**

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \cdot \varphi'(z_1)$$

5. **First-layer parameters:**

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1} \cdot x, \quad \frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1}$$

Key points: output-layer gradients are computed **first and directly** from the error; hidden-layer gradients arrive only **indirectly**, via the propagated signal — the hidden layer never sees the error directly. Every deep network is this same pattern repeated layer after layer.

5.5 Loss Functions

Mean Squared Error (MSE) — Regression

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Smooth, always non-negative. Squaring punishes large errors far more than small ones. Default for regression (prices, temperature, sensor values, autoencoder reconstruction).

Cross-Entropy — Classification

$$L_{\text{CE}} = -\log(\hat{p}_c) \quad \text{for true class } c$$

Combined numerically-stable form (what PyTorch/TensorFlow implement internally):

$$L = -z_c + \log \sum_j e^{z_j}$$

Measures how “surprised” the model is that the true class occurred. High probability on the correct class → small loss; low probability → large loss. (*softmax defined in W4*)

Why MSE fails for classification:

1. Class labels have no numeric distance — “Class 2” is not twice “Class 1”.
2. MSE does not punish **confidently wrong** predictions strongly enough.
3. Combined with sigmoid/softmax, MSE gradients shrink to almost nothing when the activation saturates — no corrective signal exactly when the model is most wrong.

Situation	MSE gradient	Cross-entropy gradient ($\hat{p} - y$)
Unsure, $\hat{p} \approx 0.5$	Moderate	Moderate — healthy push
Confidently wrong, $\hat{p} \approx 0.01$	Almost zero (saturation)	Very large, strongly corrective
Confidently correct	Small	Small

Key result: softmax + cross-entropy gives gradient = $\hat{p} - y$ with **no extra shrinking multiplier**, so it aggressively corrects confident mistakes. This is why classifiers with cross-entropy converge faster and more reliably than the same model with MSE.

5.6 Vanishing and Exploding Gradients (Canonical Treatment)

Why Depth Causes This

For an early layer, the gradient is a **product** of many local derivatives:

$$\frac{\partial L}{\partial w^{(1)}} = \frac{\partial L}{\partial a^{(L)}} \cdot \prod_{l=2}^L \frac{\partial a^{(l)}}{\partial a^{(l-1)}} \cdot \frac{\partial a^{(1)}}{\partial w^{(1)}}$$

- Most local derivatives < 1 → gradient shrinks **exponentially** → **vanishing gradients**.
- Most local derivatives > 1 → gradient grows **exponentially** → **exploding gradients**.

This is a mathematical consequence of depth, not a bug.

Vanishing gradients	Exploding gradients
Cause: saturating activations (sigmoid, tanh), or weights initialised too small	Cause: weights too large, or activations amplifying signals
$0.5^{20} \approx 10^{-6}$ (almost zero)	$1.5^{20} \approx 3300$ (huge)

Vanishing gradients	Exploding gradients
Symptom: early layers barely change; loss decreases painfully slowly; training plateaus early	Symptom: loss spikes or becomes NaN; weights grow huge; updates look chaotic

Role of activations: sigmoid's derivative never exceeds 0.25 and nears zero at saturation → strongly encourages vanishing. Tanh is centred but still saturates. ReLU's derivative is 0 or 1, so it does not shrink gradients by itself — but very large weights can still explode even with ReLU.

Common failure mode: a deep network's early layers never truly learn, so it behaves like a shallow network despite having many layers.

5.7 Weight and Bias Initialization

Why Zero Initialization Fails — Symmetry Breaking

If all weights start at zero, every neuron in a layer gets the same input, produces the same output, and receives the same gradient. They stay **identical forever** — the network can never learn different features. **Weights must never be initialised to zero.** Bias is different (it does not multiply inputs, so no symmetry problem): $b = 0$ is standard, though ReLU nets sometimes use a small positive bias to avoid dead neurons.

Naive random init is also risky: with variance too small, activations and gradients shrink across layers; too large, they explode. Variance keeps multiplying with depth.

Xavier and He Initialization

Both keep the **variance of activations and gradients stable across all layers**, for both forward and backward passes.

$$\text{Xavier / Glorot : } \text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}} \quad \text{He : } \text{Var}(W) = \frac{2}{n_{\text{in}}}$$

He uses a larger variance because ReLU zeroes roughly half its inputs, halving the effective signal variance.

Method	Best suited for	Variance
Xavier / Glorot	Sigmoid, tanh	$2/(n_{\text{in}} + n_{\text{out}})$
He	ReLU and variants	$2/n_{\text{in}}$

5.8 Mitigating Vanishing / Exploding Gradients

The Four Categories — Used Together

1. **Weight initialization** — Xavier (sigmoid/tanh) or He (ReLU).
2. **Activation choice** — ReLU family; derivative is exactly 1 for positive inputs, so gradients pass cleanly.
3. **Gradient clipping** — caps runaway gradient magnitude. Widely used in RNNs, LSTMs, Transformers. (*full treatment in W6*)
4. **Normalization layers** — BatchNorm (per mini-batch) or LayerNorm (per feature, preferred in Transformers) keep activations and derivatives in a healthy range. (*W7*)

MUST REMEMBER

- Training = forward pass $\rightarrow$ loss $\rightarrow$ gradient computation (backprop) $\rightarrow$ parameter update. **Backprop only computes gradients.**
- A gradient is a **responsibility signal**: sign = direction, magnitude = strength.
- Computational graphs make backprop systematic: global gradient = **product of local gradients**; the forward pass caches values reused backward.
- Output-layer gradients are computed first and directly; hidden-layer gradients arrive only via propagation.
- MSE for regression; **softmax + cross-entropy** for classification, because its gradient $\hat{p} - y$ does **not** vanish under saturation, unlike MSE's.
- Vanishing/exploding gradients come from **repeated multiplication of local derivatives with depth** — a mathematical consequence, not a bug.
- Zero weight init fails by **symmetry breaking**. Xavier suits sigmoid/tanh ($2/(n_{\text{in}} + n_{\text{out}})$); He suits ReLU ($2/n_{\text{in}}$).
- Stable training = good init + ReLU-family activations + gradient clipping + normalisation layers, **used together**.

6 Optimisation Algorithms

6.1 What is Optimisation?

Optimisation — adjusting weights and biases so the loss becomes as small as possible — a search over a very complicated, high-dimensional loss landscape with valleys, hills, plateaus, ridges and saddle points.

The optimiser controls: **step size** (learning rate), **directional smoothing** (momentum), **per-parameter adjustment** (adaptive methods), and **stability** (clipping, normalisation). Together these decide convergence speed, final accuracy, and training stability.

6.2 Gradient Descent

Gradient Descent Update Rule

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

θ = parameter, η = learning rate, $\frac{\partial L}{\partial \theta}$ = gradient (the slope). We always move **opposite** to the gradient because the gradient points toward steepest **increase**.

Learning rate	Behaviour
Too small	Tiny steps; extremely slow. Stable, but wastes time and compute.
Too large	Overshoots the minimum, bounces back and forth; loss oscillates or diverges.
Well chosen	Smooth, efficient progress downhill.

6.3 Variants of Gradient Descent

The three variants differ **only** in how much data is used per update.

Method	Data per update	Trade-off
Batch GD	Whole dataset	Stable and correct direction, but very slow and memory-heavy. Almost never used except on tiny datasets.
Stochastic GD (SGD)	1 sample	Very fast per step; noise helps escape saddle points/sharp minima. But noisy, unpredictable convergence. Used for online/streaming learning.
Mini-batch GD	32–512 typically	Best of both: good GPU parallelism, reasonably accurate gradient, just enough noise to help generalisation. The practical default.

Batch size effect: small batches (16–32) add noise → help escape bad regions, improve generalisation. Large batches (1024+) give smoother gradients but may converge to sharp minima and need LR tuning or warm-up.

6.4 Momentum

Plain GD **zig-zags** in narrow valleys: the gradient keeps pointing sideways across the valley walls, wasting time correcting direction instead of progressing.

Momentum Update Rule

$$v_t = \beta v_{t-1} + \nabla_{\theta} L, \quad \theta \leftarrow \theta - \eta v_t$$

v = velocity (running average of past gradients); $\beta \approx 0.9$ (90% of previous direction retained).

Effects: smooths oscillating gradients; builds speed when gradients keep pointing the same way; carries the update forward even when gradients temporarily go small.

Benefits: accelerates convergence in long narrow valleys; reduces zig-zag; helps escape shallow local minima; more stable training. Many adaptive optimisers embed a form of momentum.

6.5 Why Adaptive Optimizers Were Needed

Plain SGD uses **one global learning rate** for every parameter, but parameters behave very differently.

Problem	Explanation
Uneven parameter sensitivity	One shared LR makes some parameters learn too slowly and others become unstable; worsens with depth.
Sparse gradients	Common in NLP/recommenders where features are mostly zero. Those weights get gradients rarely → learn far slower.
Saddle points and plateaus	Gradient near zero in some directions, steep in others. An LR safe in one direction is wrong in another.

Solution: give each parameter its **own effective learning rate** from its own gradient history — dampen noisy parameters, boost rarely-updated ones, accelerate stable directions.

6.6 RMSProp

RMSProp

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$$

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{E[g^2]_t + \varepsilon}} g_t$$

Typical defaults: $\beta = 0.9$, $\varepsilon = 10^{-8}$.

- Recent gradients **large** → denominator grows → **smaller** effective step (dampens instability).
- Recent gradients **small** → denominator shrinks → **larger** effective step (boosts slow learning).
- Reduces oscillation in steep directions, handles noisy mini-batch gradients, speeds up sparse parameters, needs little tuning → popular for RNNs and early deep models.
- Limitation:** no explicit momentum, so it can still move slowly where gradients are consistent but small. This motivated Adam.

6.7 Adam (Adaptive Moment Estimation)

Adam = **momentum (direction)** + **RMSProp-style adaptive scaling (step size)**.

Adam

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t \quad (1\text{st moment — direction})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2 \quad (2\text{nd moment — step size})$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{bias correction})$$

$$\theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}}$$

Bias correction is needed because both averages start at zero and would otherwise be systematically underestimated early in training.

PROS

- Handles noisy mini-batch gradients.
- Works well with sparse gradients.
- Minimal hyperparameter tuning.
- Converges quickly across many architectures.

CONS

- Can converge to **sharp minima** → may generalise worse than SGD.
- Can **hide** a poor learning-rate choice, making debugging harder.

Adam is a strong choice when...	Adam may not be ideal when...
Training is difficult/unstable; architectures are deep or complex; gradients are sparse or noisy; early experimentation needing fast feedback with minimal tuning.	Final generalisation is critical (sharp-minima risk); fine-tuning a pre-trained model; a well-conditioned problem — here SGD + momentum often reaches better final accuracy.

The standard effective strategy: start with **Adam** to reach a good region of the loss landscape quickly; once training stabilises, switch to **SGD with momentum** (often with an LR schedule) to fine-tune and improve final generalisation.

Common mistakes: using Adam blindly for every task; never adjusting the LR; using Adam for final fine-tuning without checking; assuming Adam always beats everything else.

6.8 Learning Rate Scheduling

A **fixed** LR assumes the same step size is right at every training stage. Early training is far from the optimum with large gradients (bigger steps help); late training is near a minimum with small gradients (large steps now cause oscillation). A fixed LR forces a bad compromise: fast-but-poor-finish, or stable-but-extremely-slow.

Schedule	Behaviour and notes
Step decay	LR constant for a while, then drops sharply at predefined epochs (e.g. $\times 0.1$). Simple, but needs manual tuning of when/how much.
Exponential decay	LR multiplied by a fixed factor each step/epoch — smooth continuous reduction. Risk: too aggressive $\rightarrow$ LR too small too early.
Cosine annealing	LR follows a cosine curve: high $\rightarrow$ smoothly decreasing $\rightarrow$ very small at the end. Very popular for large models; avoids sudden changes; often used with warm restarts.
Warm-up	Start with a very small LR, increase gradually over a few epochs, then switch to a decay schedule. Prevents unstable early updates; important for large batches and Transformers.

Common pattern: short warm-up $\rightarrow$ main decay schedule (cosine or step). Stability at the start, fast progress in the middle, precise fine-tuning at the end.

	Adaptive (RMSProp, Adam)	Scheduled (step/cosine/warm-up)
How it works	Optimizer auto-adjusts step size per parameter from gradient history	We explicitly change the global LR over time by a predefined plan
Strength	Powerful early; handles noisy/sparse gradients; little tuning; robust to varying gradient magnitudes across layers	Precise control of step size; good for careful late-stage fine-tuning; often improves final generalisation
Limitation	Can converge to sharp minima; less explicit control late in training	Requires manual design/tuning; does not adapt per parameter; sensitive to initial LR

Rule of thumb: adaptive when training is difficult/noisy/exploratory; scheduled when training is stable and final performance matters most. Many pipelines combine both.

6.9 Gradient Clipping (Full Treatment)

What Gradient Clipping Does

Before applying an update, check the gradient size. If within range, leave it. If too large, **scale it down**. Ensures no single update can be catastrophically large.

- **Clipping by value** — limits each individual gradient component to a fixed range. Simple, but can distort the gradient **direction**.
- **Clipping by norm** — scales the whole gradient vector if its norm exceeds a threshold. More common, because it **preserves direction** while limiting magnitude.

It is a safeguard, not a cure-all: it does not improve generalisation, does not fix vanishing gradients, and does not replace good LR choice or good initialization. Most useful for RNNs, very deep networks, and large learning rates.

6.10 Common Optimisation Pitfalls

Pitfall	Description
Saddle points	Gradient near zero but not a minimum (downhill in some directions, uphill in others). In high dimensions, far more common than bad local minima. Optimizers stall — looks like convergence when it is not.

Pitfall	Description
Plateaus	Extremely flat regions; tiny gradients, very slow learning. Common in deep nets and with saturating activations.
Noisy gradients	Mini-batch randomness. Helps escape saddle points, but makes the loss fluctuate and convergence harder to read.
Poor conditioning	Very different curvature in different directions → zig-zag updates, slow progress, high LR sensitivity. Fixed by momentum, adaptive methods, normalisation.

Diagnosing the Real Problem

- Loss does **not decrease** or behaves erratically → an **optimisation** issue (fix: LR, optimizer, momentum, clipping, initialization).
- Loss **decreases but performance stays poor** → a **model capacity or data** issue.

Reading loss curves: smooth convergence (large batches) is easy to interpret but can settle in sharper minima. Noisy convergence (small batches, higher LR) is **normal, not failure** — noise helps escape saddle points and often finds flatter, better-generalising minima. Judge by the **long-term trend**, not short-term spikes.

MUST REMEMBER

- Optimisation decides **how to use** gradients; it navigates a high-dimensional loss landscape.
- Update: $\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$. LR too big → diverges; too small → painfully slow.
- Batch GD (whole dataset, stable/slow), SGD (1 sample, fast/noisy), **Mini-batch GD (32–512, the practical default)**.
- Momentum ($v \leftarrow \beta v + \nabla L$, $\beta \approx 0.9$) smooths zig-zag and builds speed in consistent directions.
- RMSProp adapts the LR **per parameter** from squared-gradient history. **Adam = momentum + RMSProp + bias correction** — fast, low-tuning, but can favour sharp minima; often switch to SGD+momentum for fine-tuning.
- Schedules (step, exponential, cosine annealing, warm-up) match step size to the training phase; adaptive is best early/noisy, scheduled best for stable precise finishing. Many pipelines combine both.
- Gradient clipping (by value or **by norm**) guards against exploding gradients **only**.
- Main pitfalls: **saddle points** (far more common than local minima in high dimensions), plateaus, gradient noise, poor conditioning.
- Diagnose: loss not decreasing → optimisation issue; loss decreasing but performance poor → capacity/data issue.
- Noisy loss curves are often **normal and beneficial** — judge by long-term trend.

7 Regularisation and Generalisation

7.1 Generalisation

- **Training error** — how well the model fits the data it was trained on.
- **Generalisation error** — how well the model performs on unseen data (validation/test).

A model can have very low training error and high generalisation error. Modern networks have such high capacity that they **can fit pure noise** — so good training performance alone guarantees nothing.

7.2 Underfitting, Overfitting, Capacity

Situation	Training error	Generalisation error
Underfitting	High	High
Good fit	Low	Low
Overfitting	Low	High

- **Underfitting** — model too simple for the task. Causes: insufficient capacity, overly restrictive assumptions, inadequate features, **excessive regularisation**.
- **Overfitting** — model too complex relative to the data. Memorises noise and quirks of the training set instead of general patterns.
- **Model capacity** — expressive power. The **same** model can underfit or overfit depending on the data — these are properties of the model-and-data **combination**, not the model alone.

7.3 Bias-Variance Trade-off

Definitions

- **Bias** — error from overly simple assumptions. A high-bias model is too rigid; it makes systematic errors no matter how much data it sees. → **underfitting**.
 - **Variance** — sensitivity to the specific training data. A high-variance model changes a lot with slightly different data and fits noise. → **overfitting**.
- Complexity **up** → bias down, variance up. Complexity **down** → bias up, variance down.
 - Underfitting = high bias, low variance. Overfitting = low bias, high variance.
 - Good generalisation lies in between: flexible enough for real patterns, stable enough to ignore noise.

7.4 Detecting Overfitting

Three Key Symptoms

1. **Growing train-validation gap** — training loss keeps decreasing but validation loss stops improving or starts rising.
2. **Unstable validation metrics** — validation accuracy fluctuates a lot between epochs; small data/seed changes give noticeably different results.
3. **Poor behaviour on shifted inputs** — highly confident on training-like inputs, poor on slightly different ones. This reflects **memorisation**, not learning.

Conditions that raise the risk: high-capacity model on a small/noisy dataset; training too many epochs without monitoring; complex architectures with no constraints.

7.5 L1 and L2 Regularisation

Regularised Objective

$$L_{\text{total}} = L_{\text{data}} + \lambda \Omega(w)$$

λ controls the trade-off: larger λ → simplicity; smaller λ → fit the data.

$$\text{L2 penalty : } \Omega(w) = \sum_i w_i^2 \quad \text{L1 penalty : } \Omega(w) = \sum_i |w_i|$$

Geometric picture. The unregularised loss forms concentric ellipses. L2 restricts the solution to a **circle**; because the boundary is smooth (no corners), L2 shrinks weights continuously toward zero but almost never

to zero. L1's region is a **diamond** with sharp corners on the axes; the ellipse very often first touches a **corner**, driving some weights **exactly** to zero.

Aspect	L2 (Weight Decay)	L1 (Sparsity)
Penalty	$\sum w_i^2$ (squared)	$\sum w_i $ (absolute)
Constraint shape	Circle (smooth)	Diamond (sharp corners)
Effect on weights	Shrinks smoothly toward zero	Drives some weights exactly to zero
Result	Smoother, more stable models; less noise-sensitive	Sparse model; automatic feature selection
Typical use	Most common regulariser in deep nets	When sparsity / feature selection is desired

7.6 Dropout

Co-adaptation — neurons relying heavily on other specific neurons, making the model fragile. Dropout removes this reliance.

Dropout

During training, a **random subset** of neurons is deactivated for each mini-batch; a different subset each iteration. Dropped neurons take no part in the forward or backward pass for that step.

$$\tilde{y} = \frac{m \odot y}{p}$$

y = normal output, m = random binary mask (1 = keep, 0 = drop), p = keep probability.

- Forces **redundant, distributed** representations — no fixed combination of features can be relied on.
- Also injects noise, which itself acts as a regulariser.
- **Training vs. inference:** applied **only during training**; at test time all neurons are active. Scaling is done during training (**inverted dropout**) to keep expected activations consistent.
- **Practical rates:** 0.1–0.3 for input layers, 0.3–0.5 for hidden layers. Less effective in very small networks; the rate needs tuning like any hyperparameter.

7.7 Batch Normalisation

Batch Normalisation

For each mini-batch compute mean μ_B and variance σ_B^2 of the activations, then:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y = \gamma \hat{x} + \beta$$

ε = small stability constant. γ (scale) and β (shift) are **learnable** — they let the network undo the normalisation if needed, so BN stabilises training without limiting what the network can represent.

Benefits: stabilises the distribution of layer inputs; allows **higher learning rates**; reduces sensitivity to weight initialisation; speeds up convergence; reduces “internal covariate shift”.

Why it also regularises: it was not designed as a regulariser, but mini-batch statistics differ slightly batch to batch → this injects noise → an **implicit** regulariser, similar in spirit to dropout. It can therefore reduce the need for other regularisation.

7.8 Early Stopping

How Early Stopping Works

1. Continuously monitor performance on a **validation set** during training.
2. When validation performance stops improving for a set number of epochs (the **patience**), stop.
3. **Restore the parameters from the best-validation epoch**, not the last epoch.

Training loss decreases steadily; validation loss decreases, hits a minimum, then rises as overfitting sets in. Early stopping picks that minimum. **Why it regularises:** it limits how complex the learned model can become, prevents weights from growing too large, and reduces memorisation of noise.

Practical notes: use patience so minor fluctuations do not trigger a stop; make sure the validation set is representative; needs no architecture change; combines well with other regularisers.

7.9 Data Augmentation

Data augmentation — artificially increases the size and diversity of the training set by applying **label-preserving transformations**. Each transformed sample is treated as a new example.

Data type	Common transformations
Images	Rotation, flipping, cropping, colour/contrast/exposure adjustment, blurring
Text	Synonym replacement, paraphrasing
Audio	Noise injection, time shifting

The model learns **invariance** to small realistic changes. Must be applied carefully — augmentations that do not reflect real-world variation confuse the model and hurt performance. Domain knowledge decides what is appropriate.

7.10 Implicit Regularisation from the Training Setup

Choice	Effect on generalisation
Batch size	Smaller batches add gradient noise → help avoid sharp minima → often generalise better. Larger batches are smoother/more stable but may generalise less well.
Learning rate	Very large → unstable training. Very small → fits training data extremely precisely, raising memorisation risk. Must be considered together with the number of epochs.
Model capacity	Too little → underfit; too much → overfit. Strategy: start simple, increase complexity only if needed.

7.11 Combining Regularisation Methods

Common combination	Why
L2 + data augmentation	Controls complexity while increasing data diversity
BatchNorm + moderate dropout	Used together but carefully — BN already adds noise
Early stopping + LR scheduling	Both control effective training duration
Smaller batch size + L2	Noise plus explicit weight constraint

Pitfalls When Combining

- BatchNorm already introduces noise → reduces the need for strong dropout.
- Very strong dropout + very small batch sizes → unstable training.
- Too many regularisers at once → **underfitting**.
- **Goal: balance regularisation strength — not maximise it.**

Recommended approach: start from a simple baseline; add **one** technique at a time; monitor validation performance; adjust strength gradually.

7.12 Practical Checklist for Preventing Overfitting

4-Step Checklist

1. **Diagnose first** — compare training vs. validation. Look for a large gap, rising validation loss, or highly unstable validation. Always ask “underfitting or overfitting?” **before** changing anything.
2. **Check the data** — is it large and diverse enough? Is the validation set representative? Any data leakage? If limited, consider augmentation.
3. **Revisit model complexity** — too large for the data? Reduce depth/width or add L2/dropout. Goal: the **simplest** model that performs well.
4. **Check training configuration** — batch size too large? LR too small? Too many epochs? Is early stopping enabled with validation monitoring?

MUST REMEMBER

- Generalisation = performing well on **unseen** data. Training error $\neq$ generalisation error.
- Underfitting = **high bias**, high train + high validation error. Overfitting = **high variance**, low train + high validation error.
- Complexity up $\rightarrow$ bias down, variance up (and vice versa) — the bias-variance trade-off.
- L2 shrinks weights **smoothly** toward zero (circular constraint); L1 drives some weights **exactly** to zero (diamond constraint $\rightarrow$ sparsity/feature selection).
- Dropout randomly deactivates neurons during training to stop **co-adaptation**; the full network is used at inference (inverted dropout scales during training).
- BatchNorm normalises activations per mini-batch with learnable γ, β ; stabilises training, allows higher LR, and acts as an **implicit** regulariser via mini-batch noise.
- Early stopping halts at the **best-validation** epoch (with patience) and restores those weights.
- Data augmentation = **label-preserving** transformations to increase diversity.
- Batch size, learning rate and model capacity regularise **implicitly** through the training setup.
- Combine regularisers **incrementally**; more is not better — too many at once causes underfitting. **Diagnose before intervening.**

8 Convolutional Neural Networks

8.1 Why Fully Connected Networks Fail on Images

Two Big Problems

1. **No use of spatial structure** — the model does not know two pixels are neighbours. A pattern shifted a few pixels looks like a totally different input.
2. **Parameter explosion** — every pixel connects to every neuron → an enormous number of weights → needs huge data and compute.

Nearby pixels are highly correlated, meaningful patterns are **local**, and the same pattern **repeats** at many locations. A good image model must exploit **spatial locality** and **repetition** — exactly what CNNs do.

8.2 The Convolution Operation

Convolution — a local pattern-matching operation. A small grid of learnable weights (a **filter / kernel**) is applied to small regions of the input, one region at a time. The **same** filter is reused at every location — this is **weight sharing**.

Convolution at One Location

$$y = \sum_i \sum_j w_{ij} x_{ij}$$

Each filter value is multiplied with the pixel under it; all products are summed into **one** output number = how strongly the filter matches that local region.

How it slides: start at the top-left block → compute one output → slide one step right → repeat → when out of room, move down a row and continue from the left. Filter weights never change while sliding. Collecting all outputs gives the **feature map**.

Why Convolution is Powerful

1. **Local connectivity** — captures spatial structure.
2. **Weight sharing** — drastically fewer parameters than a fully connected layer.
3. **Translation robustness** — the same filter applies everywhere, so a pattern is detected wherever it appears.

8.3 Filters and Feature Maps

- **Filter** — a small set of **learnable** weights defining a pattern the network searches for. Learned by backpropagation from random initial values — never hand-designed.
- **Feature map** — the grid of outputs from convolving **one** filter across the whole input. A **spatial response map**: high values = pattern strongly detected there.

Key Rule

One filter produces exactly one feature map. Feature maps are **learned representations**, not images — they answer “how much does my learned pattern show up here?”

Depth	What filters learn
Early layers	See raw pixels → simple fundamental patterns: edges, corners, basic colour transitions.
Middle layers	Textures, repeated motifs.
Deeper layers	See feature maps (not pixels) → object parts and complex structures.

8.4 Channels and Multiple Filters

Convolution with Multiple Input Channels

An image is a 3D volume: height × width × channels (grayscale = 1, RGB = 3). A filter **spans all input channels** — separate weights per channel. At each location it convolves independently per channel,

then **sums across channels** into one output number. **Still one filter → one feature map**. Multiple channels change **how** the filter computes, not how many maps it produces.

A single filter detects one pattern type, so layers use **many** filters; each produces its own feature map, and the layer's output is a **stack** of feature maps.

Depth in a CNN

Depth here = **number of feature maps** at a layer (not the number of layers). Going deeper into a CNN:

- Spatial dimensions (height, width) usually **decrease**.
- Depth (channels/feature maps) usually **increases**.

A trade of precise spatial detail for richer, more abstract representation.

8.5 Stride, Padding, Pooling

Control	Effect
Stride	How far the filter moves each step. Stride 1 = dense sampling, larger output. Larger stride = skips positions, smaller output, less compute, less spatial detail. Example: 7×7 input, 3×3 filter → 5×5 at stride 1, but 3×3 at stride 2.
Padding	Adds extra pixels (usually zeros) around the border, so the filter applies uniformly and border information is not lost. Controls output size. Choices: no padding (shrinks) or same padding (output size = input size).
Pooling	Summarises a small local region into one value. No learnable parameters ; applied independently to each channel. 2×2 pooling with stride 2 halves height and width, depth unchanged.

- **Max pooling** — takes the maximum value in the region. **Average pooling** — takes the mean.
- Pooling makes the network robust to **small spatial shifts**: it emphasises **whether** a pattern is present over its exact location.

Pooling	Strided convolution
Fixed operation (max/average), no learning	Learns new filter weights while subsampling
Adds robustness / invariance	Combines down-sampling with feature learning

8.6 The Standard CNN Block and Full Pipeline

The Standard Repeating Block

Convolution → Batch Normalisation → Activation → (sometimes) Pooling

- **Activation (ReLU / Leaky ReLU)** — without it, stacked convolutions collapse into one linear transform.
- **BatchNorm** — normalises activations per mini-batch → faster, more stable training. Placed **after convolution, before activation**. Regularising side-benefit. (*formula in W7*)
- **Dropout** — used mostly in the **fully connected** layers of a CNN; used cautiously in conv layers (can disrupt spatial feature learning).

End-to-End CNN Pipeline

1. Input image enters the network.
2. Conv + BN + Activation (repeated) — detect local patterns (edges, corners, textures).
3. Pooling — reduce spatial size, keep the important information, add shift robustness.
4. Steps 2–3 repeat, building increasingly abstract, less spatially precise feature maps.
5. **Flatten** — reshape the final feature maps ($H \times W \times C$) into one 1D vector. Pure structural step, no learning.
6. **Fully connected layers** — combine all features, learn **global** decision boundaries, output the prediction.

In short: convolutional layers = **feature extractor**; fully connected layers = **final reasoning / classifier**.

Shape Arithmetic Through a Block

Input $H \times W \times C$, apply F filters of size 3×3 , stride 1, padding 1:

- After convolution: $H \times W \times F$ — spatial size preserved, depth becomes F .
- After 2×2 pooling with stride 2: $\frac{H}{2} \times \frac{W}{2} \times F$ — spatial size halved, depth unchanged.

Design trade-offs: more filters $\rightarrow$ richer patterns, higher compute. Larger stride / aggressive pooling $\rightarrow$ faster and cheaper, but coarser spatial detail.

8.7 Evolution of CNN Architectures

Architecture	Key idea and contribution
LeNet	One of the earliest successful CNNs (handwritten digits). Alternates convolution and pooling, then FC layers. Proved conv + pool + FC can be trained end to end as one model. Held back by small datasets and limited compute.
AlexNet	Same structure, much larger scale . Turning point: showed CNNs scale to real-world image datasets and beat traditional CV by a wide margin. Key ingredients: heavy ReLU use, GPUs , dropout .
VGG	Very uniform design: repeatedly stack small 3×3 convolutions + pooling at regular intervals. Lessons: depth matters , and stacking small filters beats fewer large filters . Exposed the degradation problem — adding layers sometimes increased training error.
ResNet	Skip / shortcut connections let information bypass layers; a block learns only the residual (how output should differ from input). Gradients and information flow directly $\rightarrow$ reduces vanishing-gradient risk $\rightarrow$ networks of dozens/hundreds of layers become trainable. Lesson: architectural design, not raw depth, makes very deep networks work.
MobileNets	For resource-constrained deployment. Separates spatial computation from channel-wise computation , breaking one expensive convolution into much cheaper operations. Enables CNNs on phones and edge devices.
EfficientNet	Studies how models should be scaled . Scales depth, width and input resolution together in a balanced, principled way , achieving strong accuracy with fewer parameters and less compute for a given budget.

MUST REMEMBER

- FC networks fail on images: they ignore **spatial structure** and suffer **parameter explosion**. CNNs fix this with **local connectivity** and **weight sharing**.
- Convolution = local pattern matching: a filter slides over the input, multiplying and summing at each location to build a **feature map**.
- Filters are **learned** pattern detectors: edges/corners early, textures/parts/objects deeper.
- Multi-channel: one filter spans **all** channels and sums across them — **still one filter $\rightarrow$ one feature map**. Multiple filters $\rightarrow$ multiple feature maps $\rightarrow$ depth.
- Going deeper: spatial size **decreases**, depth (channels) **increases**.
- Stride controls sampling density/output size; padding controls border handling and output size; pooling (max/avg) reduces spatial size and adds shift robustness with **no learned parameters**.
- Standard block: **Conv $\rightarrow$ BatchNorm $\rightarrow$ Activation $\rightarrow$ (Pooling)**. Flatten converts feature maps into a vector; FC layers give the final prediction.
- Architecture evolution: LeNet (proof of concept) $\rightarrow$ AlexNet (scale + ReLU + GPU + dropout) $\rightarrow$ VGG (uniform depth via small 3×3 filters, exposed the degradation problem) $\rightarrow$ ResNet (**skip connections**) $\rightarrow$ MobileNet/EfficientNet (efficiency and balanced scaling).

9 Sequence Models: RNN, LSTM, GRU, Attention

9.1 Why We Need Sequence Models

Sequential data — arrives in an **ordered** stream where each element depends on what came before. Text, speech, time series, user behaviour. Changing the order can completely change the meaning.

Sequence model — a network that processes inputs one step at a time and keeps an internal **state** summarising everything seen so far. The output depends on the current input **and** the history.

Task category	Example
Sequence-to-one	Whole sequence → one output (sentiment analysis, time-series forecasting)
Sequence-to-sequence	One sequence → another (machine translation, speech recognition)
Sequence labelling	Every element gets its own label (part-of-speech tagging)

Why Feed-Forward Networks Fail — a Structural Problem

FFNs take a fixed-length input and process every input **independently**, assuming i.i.d. data. They have **no built-in memory**, so they cannot accumulate context over time. This is **structural**, not a training problem: even a perfectly trained FFN cannot model sequences, because the architecture has no recurrence. Flattening a sequence into one fixed vector destroys order and timing information.

9.2 Recurrent Neural Networks (RNNs)

Core RNN Equations

$$h_t = \varphi(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = \psi(W_{hy}h_t + b_y)$$

x_t = input at time t , h_t = hidden state (memory), φ = usually $\tanh$, ψ = output activation (e.g. softmax).

The same weight matrices are reused at every time step.

- h_t is a compact **running summary** of everything seen so far.
- Parameter sharing** across time lets one model generalise to sequences of any length.
- Unrolling** — drawing the RNN as a chain of repeated blocks, one per time step. It is the **same** cell with the **same** weights — unrolling only makes the flow of time explicit. Useful to (1) see temporal dependencies, (2) reason about gradient flow, (3) picture step-by-step processing.

Sequence classification example — “The movie was not good.”: the hidden state accumulates neutral context, then must **remember the negation** at “not”, and combine it with “good” to reach the correct negative sentiment. The final hidden state goes through a classification layer. If the model forgets “not”, it predicts the wrong sentiment.

9.3 BPTT and the Vanishing Gradient in RNNs

Backpropagation Through Time (BPTT) — the RNN is unrolled across all time steps and error from later steps is propagated backward to earlier ones. Because weights are shared, gradients flow through many **repeated** computations.

Vanishing Gradients in RNNs — the Delta vs. Deep FFNs

During BPTT, gradients are repeatedly multiplied by the **same** weight matrices and the **same** activation derivatives at every time step. Multiplying many values < 1 drives the product toward 0.

Consequence: early time steps receive almost no learning signal → the model becomes biased toward **recent** inputs and cannot learn long-term dependencies. **Why it is worse than in FFNs:** the effective depth equals the **sequence length**, so the longer the sequence, the unavoidably worse the problem.

(general mechanism in W5)

9.4 Limitations of RNNs Beyond Vanishing Gradients

Three Persistent Structural Limitations

1. **Sequential computation (no parallelism)** — each step depends on the previous hidden state, so time steps cannot be computed in parallel → slow on GPUs, especially for long sequences.
2. **Fixed-size hidden state (information bottleneck)** — the entire history is compressed into one fixed-size vector; longer/more complex sequences lose detail.
3. **Difficulty with global / long-range context** — information from early steps must pass through many intermediate states, so directly relating far-apart elements stays hard even with LSTM/GRU.

9.5 Long Short-Term Memory (LSTM)

LSTMs add an explicit **memory cell** that carries information forward with only minimal modification, instead of being recomputed every step. Crucially, they **separate memory storage from memory updates**.

Two states per time step: the **cell state** C_t (long-term memory) and the **hidden state** h_t (current output). Three learned gates, each with a **sigmoid** (0 = block completely, 1 = let everything through).

The Three LSTM Gates

Forget gate — what to remove from the cell state:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate — what new information to add:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Cell state update — combine what we keep with what we add:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output gate — how much of the cell state becomes the visible hidden state:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t)$$

How LSTM Solves the Vanishing Gradient Problem

The cell state is updated mostly by **addition** (via the forget/input gates), **not** by repeated matrix multiplication and squashing at every step. Gradients therefore flow across many time steps with much less shrinkage, and the gates give smooth, controlled updates.

9.6 Gated Recurrent Units (GRU)

GRUs simplify LSTM: **one** combined hidden state instead of a separate cell state, and **two** gates instead of three.

The Two GRU Gates

Update gate z_t — combined role of LSTM's forget + input gates: how much past to keep vs. new information to bring in.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

Reset gate r_t — how much of the past hidden state to use when computing the candidate. When $r_t \approx 0$, the model mostly ignores the past.

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

Candidate and final hidden state:

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Aspect	LSTM	GRU
States	Separate cell state + hidden state	Single combined hidden state
Gates	3 (forget, input, output)	2 (update, reset)
Parameters	More	Fewer
Expressiveness	Higher — finer memory control	Slightly lower, but often close

Aspect	LSTM	GRU
Training speed	Slower	Faster, easier on small data
Best suited for	Very long / complex sequences (long-range language modelling)	Smaller datasets, shorter sequences, rapid prototyping

Rule of thumb: start with the simpler GRU; move to LSTM only if you need more expressiveness.

9.7 Attention

Instead of compressing everything into one hidden state, attention gives **direct access to all parts of the sequence**, weighting each by current relevance.

Attention — Core Idea

$$\alpha_i = \text{softmax}(\text{score}(q, k_i)), \quad \text{context} = \sum_i \alpha_i h_i$$

α_i = attention weight for position i (how relevant it is right now). The context vector is a **weighted focus over the whole sequence**, not a single fixed summary.

Why attention helps: direct access to distant elements → long-range dependencies become easy; removes the fixed-size hidden-state bottleneck; captures global relationships.

Attention vs. recurrence: RNNs rely on **memory compression** (squeeze the past into one evolving state). Attention relies on **selective access** — instead of remembering everything, the model learns **where to look**.

9.8 Transformers

Transformer — a sequence model built entirely on attention — **no recurrence, no convolution**. It processes all elements of a sequence **in parallel**.

Self-attention — lets each element directly consider **all** other elements at once, instead of passing information step by step through time. The model learns which parts are relevant to each other, capturing global dependencies directly.

Why Transformers Dominate

1. **Parallel processing** — no step-by-step dependency → far faster training on long sequences.
2. **Direct long-range modelling** — no memory-compression bottleneck; any two positions interact directly.
3. **Scales extremely well** — performance keeps improving with more data and larger models.

Aspect	RNN (incl. LSTM/GRU)	Transformer
Processing	Sequential, step by step	Parallel, all positions at once
Memory of past	Compressed into one hidden state	Directly accessed via attention
Long-range dependency	Hard — must pass through many steps	Easy — direct connection between any two positions
Training speed (long seq.)	Slow	Fast

Now used in machine translation, language modelling, speech, computer vision, and multimodal tasks.

(future trends in W13)

MUST REMEMBER

- Sequential data has **order-dependent** meaning; feed-forward nets fail because they assume i.i.d. inputs and have **no memory** — a structural, not a training, limitation.
- RNN: $h_t = \varphi(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$, with the **same shared weights at every time step** (parameter sharing).
- Trained with **BPTT**; repeated multiplication of small values across time causes vanishing gradients, biasing learning toward **recent** inputs. Effective depth = sequence length.
- Even gated models keep 3 limits: **no parallelism**, a **fixed-size hidden-state bottleneck**, and difficulty with **global/long-range** context.
- LSTM = explicit **cell state** + 3 gates (forget, input, output); the cell state is updated mostly by **addition**, so gradients survive long sequences.
- GRU = single hidden state + 2 gates (update, reset) — fewer parameters, faster, often comparable. **Start with GRU; move to LSTM if you need more expressiveness.**
- Attention replaces **memory compression** with **selective access**: a weighted focus over the whole sequence.
- Transformers are built entirely on **self-attention**, process all positions **in parallel**, and dominate modern sequence modelling.

10 Model Evaluation and Performance Metrics

10.1 Why Accuracy Alone Is Not Enough

- Deep networks are highly expressive — they can fit extremely complex patterns and even **pure random noise**, so strong training performance can be misleading.
- A model may exploit **spurious correlations** (shortcuts) — e.g. detecting “green grass” instead of “cow” — scoring high on the test set but failing in the real world.
- Neural networks output a full **probability distribution**; two models with the **same accuracy** can behave very differently in confidence. In healthcare or fraud detection, confidence matters as much as the prediction.

The Four Pillars of Neural Network Evaluation

1. **Predictive performance** — accuracy, precision, recall, F1.
2. **Generalisation behaviour** — overfitting patterns, training vs. validation curves.
3. **Confidence quality** — calibration: does confidence match reality?
4. **Robustness and uncertainty** — stability under small input changes; how the model says “I’m not sure”.

10.2 Confusion Matrix and Classical Metrics

Cell	Meaning (fraud-detection example)
True Positive (TP)	Actual fraud, correctly predicted as fraud
False Negative (FN)	Actual fraud, wrongly predicted as not fraud — a miss
False Positive (FP)	Not fraud, wrongly predicted as fraud — a false alarm
True Negative (TN)	Not fraud, correctly predicted as not fraud

Core Formulas

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad \text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

F_1 = **harmonic mean** of precision and recall — useful when classes are imbalanced and both error types matter.

Worked example (1000 predictions: TP = 60, FP = 40, FN = 20, TN = 880): Accuracy = 94%, Precision = 60%, Recall = 75%. Accuracy looks excellent while precision is poor — this is exactly why accuracy alone hides **what kind** of mistake is being made.

- **Precision/recall trade off**: a more sensitive model catches more true positives (higher recall) but raises more false alarms (lower precision).
- **Medical diagnosis** → prioritise **recall** (missing a disease is dangerous). **Spam filtering** → prioritise **precision** (blocking a real email is worse than letting spam through).

Metrics Depend on the Decision Threshold

Networks output probabilities; a threshold (commonly 0.5) turns them into labels. **The same trained model** gives very different confusion matrices — and hence very different accuracy/precision/recall/F1 — simply by changing that threshold. Metrics reflect **both** the model **and** our evaluation choices.

10.3 Loss Surfaces and Optimisation Geometry

Loss surface — describes how the loss changes as the model’s parameters vary. Each point = one setting of all weights; the height = the loss for that setting. Training moves across this surface in **parameter** space, not input space.

Point type	Definition
Local minimum	Loss increases in every direction around this point.
Saddle point	Loss goes downhill in some directions and uphill in others at the same time . In high dimensions, saddle points are far more common than bad local minima and are a bigger source of optimisation difficulty.

Steepness and dynamics: steep regions → large gradients → fast but potentially unstable updates. Flat regions → small gradients → slower but more stable updates. This is why training is so sensitive to the learning rate.

10.4 Flat vs. Sharp Minima

Aspect	Flat minimum	Sharp minimum
Geometry	Wide valley	Narrow pit / steep well
Sensitivity to small parameter changes	Low — robust	High — fragile
Typical generalisation	Better on unseen data	Prone to overfitting

What influences which minimum is reached: learning rate, batch size, and the natural noise in SGD. **Noisier** optimisation explores wider regions → biases training toward **flatter** minima. This links training configuration directly to generalisation.

10.5 Overfitting Patterns in Deep Models

The Most Reliable Overfitting Signal

Training loss keeps decreasing steadily, while validation loss improves at first then **plateaus or starts increasing**. What matters is **how the two curves move relative to each other**, not their absolute values.

- Overfitting in deep models is **progressive**, not sudden; it can appear late, develop gradually, or be partly hidden by noisy validation metrics.
- Loss is a more reliable diagnostic than accuracy:** training accuracy keeps rising while validation accuracy just plateaus or fluctuates — the divergence is much harder to see. A **rising validation loss** means the model is becoming increasingly **confident in wrong predictions**, even when accuracy looks stable.

10.6 Calibration

Calibration — honesty in confidence. If a well-calibrated model says it is 80% confident about a group of predictions, roughly 80% of those should actually be correct. Calibration is not about being **right** — it is about knowing **how sure** you should be.

- Overconfident** — predicted probabilities **higher** than true accuracy. Modern neural networks are usually overconfident.
- Underconfident** — predicted probabilities **lower** than the true accuracy justifies.

Reliability Curve — How to Build One

- Group predictions into **bins** by confidence (e.g. all predictions between 70% and 80%).
- For each bin compute the **average confidence** and the **observed accuracy**.
- Plot average confidence vs. observed accuracy per bin. A perfectly calibrated model lies exactly on the diagonal.

Expected Calibration Error (ECE)

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|$$

M = number of bins, $|B_m|$ = predictions in bin m , n = total predictions. **Lower ECE = better calibration** — the average gap between what the model claims and what is true.

Worked example: a classifier at 95% accuracy had $ECE = 4.5\%$ (0.045) — i.e. stated confidence differs from true correctness by about 4.5 percentage points on average.

10.7 Improving Calibration

Temperature Scaling

Divide the logits by a learned constant T before softmax:

$$p_i = \text{softmax}(z_i/T)$$

- $T > 1$ **softens** the distribution (reduces overconfidence). $T < 1$ **sharpens** it.
- Dividing all logits by the same constant **does not change which class scores highest** → **accuracy stays exactly the same**, only confidence changes.

Worked example: temperature scaling brought ECE from 4.5% down to 2%, with accuracy unchanged at 95%.

Ensembles — combine predictions from multiple independently trained models. Individual models are overconfident in **different, inconsistent** ways, so averaging their probabilities smooths out the extremes.

Worked example: a 5-model ensemble improved ECE only modestly (4.5% → 4.1%) versus temperature scaling's 2% — ensembling helps calibration **indirectly**, and not always as strongly as directly rescaling confidence.

10.8 Robustness

- **Perturbation** — a small, realistic change to an input (noise, measurement shift, natural variability) that does **not** change its true meaning or label.
- **Robustness** — how **stable** a model's predictions are when the input is slightly perturbed. Two inputs that mean the same thing should get (roughly) the same answer.

Three Reasons Neural Networks Are Fragile

1. They learn highly **nonlinear** decision boundaries.
2. Input spaces are **high-dimensional** — small changes along certain directions have a large output effect.
3. Standard training optimises **average** performance over the dataset, not **local stability** around each point.

Dangerously, the model often stays **highly confident while being wrong** — instability plus confidence is a particularly risky combination. (Demo: 85% confident “polar bear” → after an imperceptible perturbation, 100% confident “dishwasher”.)

Property	Question it answers
Generalisation	How well does the model perform on new, unseen examples overall ?
Robustness	How stable is the prediction in the neighbourhood around one given example ?

A model can generalise well on average and still be locally unstable. Standard test sets only check individual points and never probe their neighbourhood — so robustness issues stay invisible unless tested explicitly.

Sensitivity curve — plots accuracy (y-axis) against increasing perturbation/noise strength (x-axis).

Curve shape	Meaning
Flat	Robust model — accuracy barely drops as noise increases.
Gradual decline	Graceful degradation — performance drops slowly and predictably.
Sharp collapse	High sensitivity — a small noise increase causes a sudden, large accuracy drop. Cannot be trusted in production.

10.9 Uncertainty and Softmax Entropy

- **Confidence** — the **highest** predicted probability — how sure the model is about its top choice.
- **Uncertainty** — how **spread out** the probability mass is across all classes.

A model can be confident about the **wrong** class and uncertain about the **right** one — related but distinct ideas.

Softmax Entropy

$$H(p) = - \sum_{i=1}^C p_i \log(p_i)$$

Low entropy → probability mass concentrated on one class (confident). **High entropy** → mass spread across several classes (uncertain).

Worked demo (4-class, 86% accuracy): average entropy was much lower for **correct** predictions (≈ 0.17) than for **incorrect** ones (≈ 0.49) — entropy carries real diagnostic signal.

Important Caveat

Uncertainty is a **signal, not a guarantee**. Models can still be **confidently wrong** — low entropy does not prove correctness. This is exactly why calibration and robustness must be evaluated too.

MUST REMEMBER

- Accuracy alone is not enough: high capacity lets deep nets memorise noise and hides confidence, generalisation and robustness problems.
- The confusion matrix (TP, FP, FN, TN) underlies accuracy, precision, recall and F1 — and **all of them change with the decision threshold**.
- Loss surfaces are complex, with many minima and (far more commonly) **saddle points**.
- **Flat minima** (wide valleys, robust to small parameter changes) generalise better than **sharp minima**. LR, batch size and SGD noise decide which you land in.
- Overfitting is **progressive**; the train/validation **loss** gap is more reliable than accuracy, which can plateau while the model quietly grows overconfident.
- **Calibration** asks whether confidence is honest; measured by **ECE**, visualised with reliability curves.
- **Temperature scaling** fixes confidence **without changing accuracy**; ensembles improve calibration indirectly by averaging out individual overconfidence.
- **Robustness** (local stability under perturbations) is separate from generalisation and must be tested explicitly with **sensitivity curves** (flat = robust, sharp collapse = fragile).
- **Softmax entropy** quantifies uncertainty — low = confident, high = uncertain — but confidence is a **signal, not a guarantee**.

11 Model Diagnostics and Debugging

11.1 From Evaluation to Diagnostics

Evaluation tells us **what** went wrong (the model performs poorly). **Diagnostics** tells us **where and why** it went wrong inside the model.

Why debugging neural networks is hard: failures are often **silent** — training runs without crashing but no real learning happens. Different problems (loss plateau, unstable loss, exploding loss, a layer that stops learning) look identical from outside, and surface metrics rarely reveal the cause. But failures are not random — they leave **internal signals**.

The Three Diagnostic Signals

- **Gradients** — are learning signals **flowing** through the network?
- **Activations** — are neurons **alive and responding**, or dead/saturated?
- **Parameters** — are weights and normalisation statistics **changing in a healthy way**?

Every failure mode shows up in at least one of these three.

11.2 Gradient Flow Diagnostics

Gradient problems are usually inspected **first**, because they **propagate forward**: they affect weight updates → activations → outputs. Fixing gradient flow often resolves many downstream issues automatically.

Layerwise Gradient Norm

$$\|g^{(l)}\|_2 = \sqrt{\sum_i (g_i^{(l)})^2}$$

Track this per layer, per training step. Plot against layer depth, or as a heat map over epochs, to see whether gradients reach everywhere.

Healthy gradient flow	Unhealthy — warning signs
No layer has near-zero gradients (learning reaches every layer); gradients are not exploding; values stay in a reasonable, fairly stable range with some natural fluctuation.	Early layers with near-zero gradients; sudden gradient explosions; highly unstable gradients swinging wildly across epochs.

Pathology	Typical cause	Diagnostic signature
Vanishing gradients	Deep network + saturating activations (tanh) with standard init	Gradients extremely small in early layers, increasing only near the output — “ early layers go dark ”
Exploding gradients	Excessively high learning rate on a deep network	Gradients start small, grow rapidly in later layers, steeper near the output — “ large, unstable spikes ”
Dead gradient paths	Extremely small weight init, or a configuration/architecture error	Gradient norm exactly zero across all layers — signal completely blocked , not just decaying

Important Distinction

Vanishing gradients are a **numerical decay** problem — small but non-zero, shrinking with depth. A **dead gradient path** is a **configuration/architectural** issue — exact zeros everywhere (bad init, frozen layers, gradients explicitly blocked in the graph). (*underlying mechanism in W5*)

Key lessons: most gradient issues come from **configuration choices** (init scale, learning rate), not code bugs. Debugging should start with **diagnostics**, not with guessing fixes.

11.3 Activation Diagnostics

Dead ReLU — a ReLU neuron that always outputs 0 for all inputs. Its local gradient is 0, so its weights stop updating and it **typically cannot recover** — a permanent dead region.

Detection: track the **fraction of activations that are exactly zero** in each ReLU layer over training. A layer quickly reaching 70–80% zeros has effectively gone dead.

Common causes: high learning rates (large updates push weights into an always-negative region); poor weight initialization; large negative biases; deep networks trained without normalisation.

Saturation — sigmoid/tanh outputs cluster near the flat limits (0/1 for sigmoid, $-1/+1$ for tanh). There the activation derivative is near zero, and since backprop **multiplies** by that derivative, the gradient shrinks drastically.

Aspect	Dead ReLU	Saturated Sigmoid / Tanh
Output	Exact zero	Constant near a limit (0/1 or $-1/+1$)
Cause	Weights pushed into always-negative-input region	Large pre-activation values push outputs to the flat ends
Effect on gradient	Near-zero	Near-zero
Shared consequence	Silently reduces the model's learning capacity	Same

Monitoring Activation Distributions — What to Look For

- **Across layers** (fixed point in training): similar spread and medians, no layer showing extreme collapse or saturation → stable, balanced behaviour across depth.
- **Across time** (one layer over epochs): stable distributions, no major drift or collapse → healthy, controlled learning.
- **Fraction of zero ReLU activations:** 40–60% is **common and often healthy**. Watch the **trend**, not the absolute number — a slowly varying or stable curve is fine; a gradual, unbounded increase signals growing dead-neuron capacity loss.

Caveat: dead ReLUs can increase gradually **while training loss keeps decreasing** — the loss curve hides this completely.

11.4 Parameter Diagnostics

Why monitor weight distributions: they reveal whether learning is balanced across layers, whether weights are collapsing or exploding, and whether some layers are effectively untrained — all invisible in loss curves.

- **Healthy across layers:** similar distributions, no extreme spread or collapse, no single layer dominating in magnitude.
- **Healthy across time:** distributions for a given layer remain largely stable epoch to epoch.

Weight norm — one norm value per layer per step — a compact “heartbeat” for training dynamics.

Weight-norm behaviour	Interpretation
Flat (barely changing)	Layer may be frozen or undertrained
Rapidly / inconsistently growing	Unstable optimisation (e.g. LR too high); risk of divergence
Smooth, gradual growth	Healthy training

Note: different layers are **expected** to show different norm dynamics — e.g. an output layer (fewer parameters, directly constrained by the loss) often has a smaller, more slowly growing norm. This is normal.

Anomaly	Typical cause	Detection signature
Frozen layer	<code>requires_grad = False</code> set by mistake, or optimizer misconfiguration	Nearly flat weight norm across all epochs — no gradient updates reaching it
Exploding weights	Learning rate too high	Weight norms grow rapidly and inconsistently across layers — risk of divergence/NaN
Undertrained output layer	A much smaller learning rate assigned to that layer	That layer's norm grows much more slowly than others — a performance bottleneck

11.5 BatchNorm Diagnostics

BatchNorm layers maintain **running mean** and **running variance**. Monitoring them helps detect internal covariate shift, unstable feature distributions, or layers not learning properly — instabilities that may never show up in loss/accuracy curves.

Common Misconceptions

- BatchNorm statistics **do not** need to converge to the same value across layers.
- A small variance is **not** inherently bad.
- Different layers are **expected** to behave differently — deeper layers learn different feature scales, so differing running mean/variance magnitudes across layers is **normal**.
- These statistics are **diagnostic tools**, not pass/fail correctness tests.

11.6 A Structured Debugging Workflow

Symptom-to-Diagnostic Mapping

- Loss is **stuck** → first check **gradients**.
- Loss is **unstable** → check **weight norms** and **batch norm statistics**.
- Model **underfits** → look at **activations**.
- Something feels **strange** → inspect **parameters** for frozen or broken layers.

Case Study — Diagnosing a Broken Network

Setup: excessive depth, a very high learning rate, no normalisation.

1. **Observe:** training loss jumps up then plateaus high; gradient norm spikes early then stabilises. Flat-high loss + flat-high gradient norm ⇒ **learning rate probably too large**.
2. **Inspect gradients:** despite large early gradients, final layer-wise gradients are near zero — gradient flow has **collapsed**.
3. **Inspect activations:** dead-ReLU fraction rises sharply to 80–90% in the final layers. High LR pushed activations permanently negative — a **silent failure**: loss looks stable but learning has stopped.
4. **Inspect parameters:** weight norms are stable across layers — rules out a parameter-level anomaly as root cause.
5. **Synthesise:** high LR → unstable early updates → dead ReLUs → blocked gradient flow → gradients collapse → training stalls **without any explicit divergence**. Not detectable from the loss curve alone.
6. **Apply fix and remeasure:** reduce the learning rate (here by 100×). Loss then decreases smoothly, gradient norms decay instead of exploding, and the dead-ReLU problem resolves — confirming the root cause.

Key lessons: the learning rate is often the **first** thing to debug — one root cause behind many seemingly unrelated symptoms. High LR can **silently kill neurons**. Effective debugging combines **multiple** signals. Secondary fixes (He/Xavier init, BatchNorm, Leaky ReLU, gradient clipping) help but **none compensates for a fundamentally wrong learning rate**.

11.7 Monitoring Tools and Experiment Tracking

TensorBoard — a visualisation dashboard for training: monitor in real time, compare experiments, inspect internal behaviour. A `SummaryWriter` logs **scalars** (loss, gradient norm) and **histograms** (weight/bias distributions) to a log directory.

Common TensorBoard mistakes: watching too many signals at once; focusing on single points instead of trends; treating it as a **replacement** for debugging reasoning rather than a complement. **Good habit:** start with loss and gradients, drill deeper only if something looks wrong.

What Every Experiment Must Record

1. Model architecture
2. Dataset (and splits)
3. Hyperparameter configuration

4. Logged metrics / artefacts
5. A **unique identifier** (prevents overwriting logs, allows fair comparison)

Key principle: if an experiment cannot be reproduced, it effectively does not exist.

Common tracking mistakes: overwriting logs (no unique run ID); forgetting to record hyperparameters; relying on memory instead of a saved config file; keeping results only inside notebooks.

MUST REMEMBER

- Diagnostics looks **inside** the model for **why** it fails. Three core signals: **gradients, activations, parameters**.
- Vanishing gradients (early layers go dark), exploding gradients (large unstable spikes) and **dead gradient paths** (uniform exact zeros) each have distinct signatures, measured via layerwise gradient L2 norms.
- **Dead ReLUs** output exact zero permanently and rarely recover; **saturated sigmoid/tanh** sit near their limits — both give near-zero gradients and silently reduce learning capacity.
- Healthy activation and weight distributions look **similar across layers** and **stable across time**. 40–60% zero ReLU activation is normal — **watch the trend, not the number**.
- Weight norms are a compact “heartbeat”: **flat = frozen, rapidly growing = unstable, smooth growth = healthy**.
- Parameter anomalies (frozen, exploding, undertrained layers) are found by comparing against a **healthy baseline**.
- BatchNorm running mean/variance **differing across layers is normal**, not a bug.
- Debugging workflow: stuck loss → gradients; unstable loss → norms/BatchNorm; underfitting → activations; strange behaviour → parameters. Then: observe → gradients → activations → parameters → fix → **remeasure**.
- **The learning rate is the most common root cause**. TensorBoard gives visibility; unique run IDs + logged configs give reproducibility.

12 Hyperparameter Tuning and Experiment Control

12.1 Why Manual Tuning Fails

Training a deep model is **not deterministic**: learning rate, batch size, initialisation, data order and randomness all influence the outcome. Two people training the same model can get very different results — training a network is **running an experiment**, not just running code.

Three Reasons Manual Tuning Breaks Down

1. **Highly non-linear dynamics** — a small change in LR or batch size can completely change behaviour; intuition becomes unreliable.
2. **Combinatorial explosion** — even 4 hyperparameters with 10 candidate values each gives 10^4 combinations. Not feasible manually.
3. **Human bias** — people try familiar values, stick with early results, and miss better configurations elsewhere.

Automated search removes bias, explores systematically, and finds solutions a person would never try.

12.2 The Hyperparameters That Matter

The Four Dominant Hyperparameters

- **Learning rate** — how fast the model learns and how stable training is.
- **Batch size** — how much noise/stability is in each gradient update.
- **Model capacity (depth and width)** — how much complexity can be represented.
- **Regularisation strength** — how much the model is constrained against overfitting.

If these four are badly chosen, **no optimizer or architecture can save the model**.

They Interact — Rules of Thumb

- A **large learning rate** may require a **smaller batch size** to stay stable.
- A **large model** needs **stronger regularisation** to avoid overfitting.
- A **small dataset** requires a **smaller model**, so it does not simply memorise the data.

Tuning is about balancing interacting forces, not tuning each knob in isolation.

12.3 Learning Rate — Symptoms of a Bad Choice

The LR scales **every** change to **every** weight — even perfectly computed gradients get distorted by a bad learning rate. (*update rule in W6*)

Learning rate	Observed behaviour
Very low (e.g. 10^{-5})	Loss decreases on a very gentle slope. Stable but inefficient; can also get stuck in suboptimal solutions, especially in deep networks.
Moderate (e.g. 10^{-3})	Loss decreases smoothly and converges — healthy training .
High (e.g. 10^{-1})	Loss oscillates and struggles to converge — parameters keep overshooting the minimum.
Very high (e.g. 1–10)	Loss diverges or becomes unstable — training fails completely.

The LR interacts strongly with batch size and optimizer choice: small batches, large models and strong regularisation all make training more LR-sensitive.

Practical rule: always tune the **learning rate first**. If training diverges, reduce it. If training is too slow, increase it cautiously.

12.4 Batch Size

Batch size does **not** change the objective function — it changes **how the model moves along the loss surface**.

Batch size	Effect
Small	Noisy but frequent updates (high gradient variance). Better exploration — can escape sharp, poor regions.
Large	Smooth, stable, less stochastic updates. Can converge to sharper minima.
Full batch	Smoothest and most stable gradients, but updates computed rarely and are almost deterministic.

Key takeaway: batch size controls **gradient noise**, not the loss function. Batch size and learning rate must be tuned **together**.

12.5 Automated Hyperparameter Search

Grid search — evaluates **all** combinations of a predefined set of values (e.g. 3 LRs $\times$ 3 batch sizes = 9 runs).

Random search — samples hyperparameters **independently from distributions** rather than from a fixed grid. Key insight: not all hyperparameters matter equally, so it is more efficient to explore the important ones flexibly.

Aspect	Grid Search	Random Search
Sampling	All fixed combinations	Independent random samples from distributions
Coverage	Uniform, but wastes trials on low-impact dimensions	More diverse; focuses effectively on important dimensions
Efficiency (same budget)	Lower — many uninformative configurations	Higher — often finds better models sooner
Scalability	Poor with many hyperparameters	Scales better with limited compute

With the same budget, random search explores LR values never present in the grid and usually finds a better best-configuration — which is why it is **the default baseline in practice**.

12.6 Bayesian Optimisation

Core Idea

Treat model performance as an **unknown function** of the hyperparameters. Each training run gives one **sample** of that function. Bayesian optimisation builds a **probabilistic model** of it and uses that to decide: “given what I have seen so far, where is the most promising place to try next?”

Explore vs. exploit:

- **Exploration** — try uncertain regions of the space to discover new possibilities.
- **Exploitation** — refine around known good regions to squeeze out better performance.

This balance makes it far more **sample-efficient** than blind search — good hyperparameters in far fewer trials, which matters because deep learning experiments are expensive, noisy and highly sensitive. In practice use libraries: **Optuna**, **Hyperopt**, **Ray Tune**.

12.7 A Systematic Experimentation Workflow

The 5-Step Workflow

1. **Define the problem** — dataset, metric, what “success” means. Without this, all tuning is noise.
2. **Build a simple baseline** — confirms the pipeline works at all and gives a reference point.
3. **Decide what to tune and how** — which hyperparameters matter, what ranges, what validation strategy. This is where automated search becomes essential.
4. **Run controlled experiments** — fixed seeds, tracked parameters, saved checkpoints, logged metrics. **Most projects fail here** — without control you cannot compare or reproduce.
5. **Select the best model on validation data, then test once** — never select on the test set. Evaluate on test **only once**, for a clean unbiased estimate.

Running a professional tuning experiment: split train/validation explicitly so every configuration is compared on the same held-out data; keep the architecture **fixed** so only hyperparameters vary; define the search space as **distributions**, not fixed values; run multiple trials by sampling.

- **Early stopping** — define a **patience**; track the best validation score so far; stop if it does not improve for that many epochs. Saves compute on unpromising runs.
- **Checkpointing** — in each trial, store the model if it is the best seen so far. After all trials, the best configuration and its saved model are kept.

12.8 Reproducibility

Sources of Randomness in Deep Learning

- Randomly initialised weights.
- Training data shuffled differently each run.
- Non-deterministic GPU operations.
- Techniques like dropout that deliberately inject randomness.
- Multi-threading / parallel execution changing computation order.

Any of these can change the final model even with identical code, data and architecture. In one demonstration, the exact same code run twice gave **different** validation accuracies — a real difference caused purely by uncontrolled randomness.

Why it matters: without reproducibility you cannot trust improvements (real gain or lucky randomness?), debugging becomes very hard, and experiments cannot be fairly compared.

Reproducibility Checklist

1. **Fix all random seeds** — NumPy, PyTorch, and every other library used.
2. **Fix train/validation/test splits** — the same split every run.
3. **Log all hyperparameters** — record and reuse them exactly.
4. **Save model checkpoints** — so a specific trained model can always be recovered.
5. **Track software/hardware versions** — frameworks, drivers, hardware.
6. **Avoid mixing multiple runs in the same logs** — keep records separate.

After fixing all seeds and recreating the splits from those seeds, re-running the same experiment twice gave **exactly** the same accuracy — reproducibility, once engineered, removes run-to-run variation entirely.

MUST REMEMBER

- Manual tuning fails at scale: **non-linear dynamics, combinatorial explosion, human bias**.
- Only 4 hyperparameters dominate: **learning rate, batch size, model capacity, regularisation strength** — and they **interact**, so tune them together.
- Learning rate controls step size: too low → slow/stuck; too high → oscillation/divergence. **Tune it first.**
- Batch size controls **gradient noise**, not the objective: small = noisy but exploratory; large = smooth but can reach sharper minima.
- Grid search = exhaustive but wasteful. **Random search = same budget, better coverage of the dimensions that matter — the default baseline.**
- **Bayesian optimisation** models performance as a function of hyperparameters and balances **explore vs. exploit** to find good settings in far fewer trials (Optuna, Hyperopt, Ray Tune).
- Systematic workflow: define problem → baseline → decide search → controlled experiments → **validate, then test once.**
- Early stopping saves compute on bad runs; checkpointing preserves the best model found.
- **Reproducibility must be engineered:** fixed seeds, fixed splits, logged configs, tracked versions. If an experiment cannot be reproduced, it does not exist.

13 Explainability, Responsible AI and Future Trends

13.1 Why Explainability Matters

In high-stakes settings (healthcare, finance, hiring, legal), **why** a decision was made is as important as **what** the decision was.

The Black-Box Problem

Deep networks learn complex internal representations but do not explain themselves. A model can be **highly confident and highly accurate** while relying on patterns that are incorrect, biased, or unsafe. Because these are invisible, problems go unnoticed until real harm occurs. **High accuracy does not guarantee correct reasoning.** A model may learn a **shortcut** — e.g. judging an X-ray by the hospital's scanner watermark instead of the disease.

Explainability is a safety mechanism. It lets us inspect behaviour and debug errors, detect bias before deployment, verify reasoning against domain knowledge, and satisfy regulators who increasingly demand justifications, not just predictions.

13.2 Interpretability vs. Explainability

	Interpretability	Explainability
Nature	Intrinsic — the model is understandable by its own structure	Post-hoc / external — extra methods explain an already-trained opaque model
Transparency	Human can directly reason about how inputs affect outputs	Model stays a black box; a separate technique produces the explanation
Scope	Global understanding of the whole model	Often local , instance-level (why this one prediction)
Examples	Linear regression, small decision trees, rule-based systems	Feature attribution, saliency maps, LIME, SHAP
Cost	Usually limited model complexity/capacity	Explanation is not guaranteed to be the model's true reasoning

The choice is a **design decision**, not a preference. High-stakes domains with strict transparency rules may **require** an interpretable model; most modern applications need complex deep models, so **post-hoc explainability** is the practical route.

13.3 Explainability Methods

Feature importance — which inputs mattered most — asked **globally** (across the dataset) or **locally** (for one prediction).

Saliency map — assigns an importance score to each input pixel, highlighting the regions that most influenced a prediction. Gradient-based: if changing a pixel slightly causes a large output change, that pixel is important. Visualised as a heat map.

Saliency maps are useful for debugging (does the model focus on meaningful regions or spurious patterns?), but they show **influence, not reasoning** — noisy, unstable, and not causal proof.

Perturbation-Based Methods

Core idea: if a small change to an input feature causes a large change in the prediction, that feature must be important. Systematically perturb the input and watch the output react.

- **LIME** (Local Interpretable Model-agnostic Explanations) — explains **one** prediction. Generates many perturbed versions around that point, queries the black box on each, and fits a simple **interpretable local surrogate** (e.g. a small linear model). The explanation comes from the surrogate, not the original model.
- **SHAP** (SHapley Additive exPlanations) — rooted in **game theory**. Each feature is a “player”; SHAP computes how much each contributes **on average across all possible feature combinations**, giving strong theoretical guarantees at higher computational cost.

Aspect	LIME	SHAP
Basis	Local linear surrogate model	Game-theoretic (Shapley values)
Speed	Faster, more heuristic	Slower, more computationally expensive
Consistency	Can vary across runs	More consistent across runs and models
Theoretical grounding	Weak	Strong

Both are **model-agnostic** — they work on any black-box model. **Limitations:** computationally expensive; sensitive to how perturbations are generated; local explanations may not generalise; perturbed inputs can stray off the true data manifold and mislead.

Model-Specific Methods

- **Grad-CAM** (Gradient-weighted Class Activation Mapping) — for **CNNs**. Uses the gradients flowing into the **last convolutional layer** to find which feature maps matter for a class, combined into a heat map. Answers: “which **regions** of the image mattered for this class?”
 - **Integrated Gradients** — instead of one gradient, **accumulates gradients along a path** from a baseline input (e.g. a black image) to the actual input. Principled feature attribution, but the result **depends on the baseline choice**.
- Grad-CAM: highly visual, best for CNNs/images, **spatial-region** explanations.
 - Integrated Gradients: more general, **feature-level** attributions, baseline-dependent.
 - Both are gradient-based → noisy, sensitive to design choices, **never causal proof**. Diagnostic tools, not ground truth.

PROS

- Improve transparency and trust in regulated/high-stakes domains.
- Help debug models and identify spurious correlations.
- Give insight into why a particular prediction was made.

CONS

- Most explanations are **local**, assumption-dependent, noise-sensitive.
- Can be **unstable** across very similar inputs.
- Provide **no causal guarantees** — highlighted features are not proven causes.
- No single method gives complete transparency.

Correct use of explainability: treat it as a **diagnostic and investigative** tool, not proof. Use **multiple** methods, look for **consistent patterns** across them, treat explanations as **hypotheses**, combine with rigorous evaluation and domain expertise, and focus on **patterns**, not single examples.

13.4 Bias and Fairness

Bias in AI — **systematic differences in how a model behaves across different groups**. Not about opinions or intent. A model can be accurate overall while its errors are unevenly distributed — invisible if we look only at aggregate accuracy.

Source of bias	Description
Historical bias	Training data reflects past unfair decisions (e.g. biased hiring history), so the model learns and repeats the pattern.
Representation bias	Some groups are under-represented in the data (e.g. few images of certain skin tones), so the model performs worse for them.
Measurement bias	A proxy variable unintentionally encodes sensitive information (e.g. ZIP code encoding race or socioeconomic status).
Deployment bias	The model is used in a different context than it was trained for (e.g. trained in one country, deployed in another).

Accuracy ≠ Fairness

A medical model can be 95% accurate overall, yet if that 5% of errors falls mostly on one group (e.g. under-diagnosing a disease in women), the model is **unfair despite being accurate**. Optimising

average accuracy hides such disparities. Fairness asks **who** the model is right or wrong for, not just **how often**.

Detection: identify **protected attributes** (gender, race, ...), split the validation data by group, and compute metrics **separately per group**.

Fairness definition	Question it asks
Demographic Parity	Do different groups receive positive outcomes at similar rates ? (e.g. equally frequent loan approvals)
Equal Opportunity	Among people who truly deserve a positive outcome, are all groups treated equally?
Equalised Odds	Goes further — are both correct approvals and incorrect rejections balanced across groups?

The Impossibility Result

If two groups have **different base rates** (e.g. different disease prevalence), it can be **mathematically impossible** to satisfy demographic parity and equalised odds **simultaneously**. Fairness cannot simply be “optimised away” — it involves explicit trade-offs and value judgments.

Fairness evaluation is **not a one-time check** — it must be monitored continuously during validation **and** deployment.

Mitigation — Three Points in the Pipeline

1. **Pre-processing (fix the data, before training)** — resampling/reweighting to rebalance the dataset; correcting historically biased labels.
2. **In-processing (fix the training itself)** — add a **fairness constraint** to the loss penalising large error-rate gaps across groups; **adversarial debiasing**, training the model so another model cannot infer protected attributes from its representation.
3. **Post-processing (fix the outputs)** — different **decision thresholds per group** to equalise false-negative rates. Easier to implement, but raises transparency and policy-justification concerns.

The unavoidable trade-off: fairness interventions often **reduce overall accuracy**, and improving fairness for one group can worsen outcomes for another. There is no universal solution — mitigation requires explicit value judgments, is context-dependent, and needs conscious design, documentation, and ongoing monitoring.

13.5 Future Directions

13.5.1 Transformers and Attention Everywhere

Transformers removed the sequential bottleneck entirely by **eliminating recurrence** and using attention as the core mechanism: not all parts of the input are equally important, so the model **dynamically focuses** on the most relevant parts (“Attention is all you need”). Now used in language models, vision, audio/speech, and combined text+image+audio systems. Their other key advantage is **scalability** — parallel computation trains efficiently on massive datasets, and performance keeps improving with more data and compute, making them the foundation of large-scale foundation models. (*mechanics in W9*)

13.5.2 Diffusion Models and Generative AI

The Diffusion Idea

Take a clean image and gradually add random noise until it becomes pure static. Train a model to **reverse** this process — to slowly remove noise, step by step. Generation then means: start from pure noise and gradually denoise until a meaningful sample emerges.

Why popular: stable to train (generation is broken into many small manageable steps, unlike older failure-prone generative models) and produces high-quality, diverse samples. They power text-to-image systems, image editing, in-painting, and style transfer, and are typically **combined with transformers/attention** to condition generation on text.

13.5.3 Efficiency, Scaling, Multimodality

Force	What it means
Efficiency	Achieving more capability with less compute . A first-class design goal now that AI systems are products used by millions daily — they must respond quickly, run reliably, and be cost-efficient.
Scaling	Larger models on more data learn more general representations (scaling laws), enabling foundation models. But scaling is expensive and energy-intensive, so the field is shifting toward smarter scaling — better architectures and data quality, not just raw size.
Multimodality	Combining text, images and sound in one system — reading a document with text+images, answering across both, generating images from text.

Overall direction: AI that is not just more powerful but **efficient**, scales **responsibly**, works across modalities, and deploys reliably — with **alignment** (interpretable, fair, trustworthy) as a central concern, not an afterthought.

MUST REMEMBER

- High accuracy $\neq$ trustworthy reasoning — explainability is a **safety and risk-reduction** tool.
- **Interpretability** = intrinsic, global, transparent by design. **Explainability** = post-hoc, usually local, applied to a black box after training.
- Saliency maps (gradient-based) show **influence**, not causal reasoning.
- **LIME** = fast local linear surrogate; **SHAP** = slower, game-theoretic (Shapley values), more consistent. Both are **model-agnostic**.
- **Grad-CAM** = CNN-specific spatial heat maps; **Integrated Gradients** = path-based feature attribution from a baseline. Both gradient-based, neither is causal proof.
- Use explainability as a **hypothesis-generating diagnostic**: multiple methods, consistent patterns, domain expertise.
- **Bias** = systematic group-wise differences, usually from the data: historical, representation, measurement, deployment.
- Fairness metrics: **demographic parity**, **equal opportunity**, **equalised odds** — and with different base rates they can be **mathematically impossible to satisfy together**.
- Mitigation at **pre-processing** (data), **in-processing** (loss/training), or **post-processing** (thresholds) — always with an accuracy/fairness trade-off.
- Future: **Transformers** (attention replaces recurrence, scales), **diffusion models** (denoise from noise), and the 3 forces of **efficiency, scaling, multimodality** — with responsibility as a central concern.

14 Quick Reference Sheet

14.1 Activation Functions at a Glance

Name	Formula	Range	Use where	Main drawback
Step / sign	$\text{sign}(z)$	$\{-1, +1\}$	Perceptron only	Not differentiable; no confidence
Sigmoid	$1/(1 + e^{-z})$	$(0, 1)$	Binary output layer	Saturates $\rightarrow$ vanishing gradients; not zero-centered
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	$(-1, 1)$	Hidden layers (older nets), RNNs	Still saturates
ReLU	$\max(0, z)$	$[0, \infty)$	Default hidden layers	Dying ReLU (permanent dead neurons)
Leaky ReLU	$\max(\alpha z, z), \alpha \approx 0.01$	$(-\infty, \infty)$	When ReLUs die	Extra hyperparameter α
PReLU	Same, α learnable	$(-\infty, \infty)$	High-performance vision models	Extra parameters
Softmax	$e^{z_i} / \sum_j e^{z_j}$	$(0, 1)$, sums to 1	Multi-class output layer	Output layer only; can be overconfident
Linear	z	$(-\infty, \infty)$	Regression output layer	No non-linearity

14.2 Optimizers at a Glance

Optimizer	Update rule (core)	Strengths	Weaknesses
Batch GD	$\theta \leftarrow \theta - \eta \nabla L$ over whole dataset	Stable, correct direction	Very slow, memory-heavy
SGD	Same, one sample	Fast per step; noise escapes saddles	Very noisy, unpredictable
Mini-batch GD	Same, 32–512 samples	Practical default: fast + stable	Still one global LR
Momentum	$v \leftarrow \beta v + \nabla L; \theta \leftarrow \theta - \eta v$	Kills zig-zag; builds speed; escapes shallow minima	Extra hyperparameter β
RMSProp	Divide step by $\sqrt{E[g^2]}$	Per-parameter LR; good for RNNs, sparse gradients	No momentum $\rightarrow$ slow on consistent small gradients
Adam	Momentum + RMSProp + bias correction	Fast, low-tuning, robust; default choice	Can find sharp minima; hides bad LR

14.3 Regularisation Techniques at a Glance

Technique	Mechanism	When to use
L2 (weight decay)	Penalty $\lambda \sum w_i^2$; circular constraint; shrinks weights smoothly toward 0	Default regulariser in deep nets; smooth, stable models
L1	Penalty $\lambda \sum w_i $; diamond constraint; drives weights exactly to 0	When sparsity / automatic feature selection is wanted
Dropout	Randomly deactivate neurons each mini-batch; prevents co-adaptation	Large nets; 0.1–0.3 input, 0.3–0.5 hidden; FC layers of CNNs
Batch normalisation	Normalise activations per mini-batch, then rescale with learnable γ, β	Deep nets; allows higher LR, faster convergence; implicit regulariser
Early stopping	Stop at the best-validation epoch (with patience); restore those weights	Always — free, no architecture change
Data augmentation	Label-preserving transformations create new training samples	Small datasets; images/text/audio
Smaller batch size	Adds gradient noise $\rightarrow$ biases toward flatter minima	When generalisation matters more than throughput

14.4 Hyperparameter Cheat Sheet — Too High vs. Too Low

Hyperparameter	Too high	Too low
Learning rate	Loss oscillates, spikes, or diverges/NaN; overshoots the minimum; can silently kill ReLUs	Painfully slow; stuck in poor solutions; wastes compute
Batch size	Smooth but may converge to sharp minima; needs LR retuning/warm-up; worse generalisation	Very noisy updates; unstable; poor GPU utilisation
Model capacity (depth/width)	Overfits; memorises noise; expensive	Underfits; high bias; cannot capture real patterns
Regularisation strength λ	Underfits — model too constrained	Overfits — no constraint on complexity
Dropout rate	Underfits; unstable with small batches	Little regularising effect; co-adaptation persists
Momentum β	Overshoots; oscillates past minima	Loses the smoothing/acceleration benefit
Number of epochs	Overfits (fix with early stopping)	Underfits; training stopped too soon

14.5 Symptom → Diagnosis → Fix

Symptom	Likely cause	Fix
Loss does not decrease at all	Gradient flow blocked / dead paths; LR far too small or too large	Check layerwise gradient norms first; verify init; adjust LR
Loss decreases then plateaus high	Vanishing gradients; dead ReLUs	ReLU-family activation, He init, BatchNorm, lower LR, Leaky ReLU
Loss spikes / becomes NaN	Exploding gradients; LR too high	Gradient clipping by norm ; reduce LR; better init; normalisation
Training loss ↓ but validation loss ↑	Overfitting	Early stopping, L2, dropout, data augmentation, smaller model
Both errors high	Underfitting / too little capacity or too much regularisation	Increase depth/width; reduce regularisation; train longer
Loss fine, accuracy poor on new data	Capacity or data issue, not optimisation	More/better data; check for leakage and distribution shift
Model confident but wrong	Poor calibration	Temperature scaling (accuracy unchanged); ensembles
Prediction flips on tiny input change	Poor robustness	Perturbation tests, sensitivity curves, augmentation
One layer's weight norm is flat	Frozen layer (<code>requires_grad = False</code>) or optimizer misconfiguration	Check optimizer parameter groups and grad flags
Two runs give different results	Uncontrolled randomness	Fix all seeds, fix splits, log configs, track versions

14.6 Formula Sheet

Everything on One Page Neuron / layer

$$z = \sum_i w_i x_i + b; \quad a = \varphi(z); \quad z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

Sigmoid / tanh / ReLU / softmax

$$\sigma(z) = \frac{1}{1 + e^{-z}}; \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}; \quad \text{ReLU}(z) = \max(0, z); \quad \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Losses

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2; \quad L_{\text{CE}} = -\log(\hat{p}_c); \quad \frac{\partial L_{\text{CE}}}{\partial z} = \hat{p} - y$$

Chain rule / gradient descent

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}; \quad \theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

Momentum / RMSProp / Adam

$$v_t = \beta v_{t-1} + \nabla L; \quad E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2; \quad \theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}}$$

Regularisation

$$L_{\text{total}} = L_{\text{data}} + \lambda \Omega(w); \quad \Omega_{L2} = \sum_i w_i^2; \quad \Omega_{L1} = \sum_i |w_i|$$

BatchNorm / Dropout

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y = \gamma \hat{x} + \beta; \quad \tilde{y} = \frac{m \odot y}{p}$$

Initialization

$$\text{Xavier} : \text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}; \quad \text{He} : \text{Var}(W) = \frac{2}{n_{\text{in}}}$$

RNN / LSTM cell

$$h_t = \varphi(W_{\text{hh}} h_{t-1} + W_{\text{xh}} x_t + b_h); \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Attention

$$\alpha_i = \text{softmax}(\text{score}(q, k_i)); \quad \text{context} = \sum_i \alpha_i h_i$$

Metrics

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}; \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}; \quad F_1 = \frac{2 \text{ P R}}{\text{P} + \text{R}}$$

Calibration / uncertainty

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|; \quad H(p) = - \sum_{i=1}^C p_i \log p_i$$

Numerical stability

$$\log \sum_i e^{x_i} = c + \log \sum_i e^{x_i - c}, \quad c = \max_i x_i$$

Gradient norm

$$\|g^{(l)}\|_2 = \sqrt{\sum_i \left(g_i^{(l)}\right)^2}$$

14.7 Final Exam-Day Checklist

1. **Structure:** neuron $\rightarrow$ layer $\rightarrow$ MLP $\rightarrow$ CNN/RNN/Transformer. Every one is “linear transform + non-linearity”, repeated.
2. **Why non-linearity:** without it, any depth collapses into one linear model.
3. **Why depth:** hierarchical feature reuse — far more parameter-efficient than width (UAT caveat).
4. **Training loop:** forward $\rightarrow$ loss $\rightarrow$ backprop (gradients only) $\rightarrow$ optimiser update.
5. **Backprop = chain rule on a computational graph.** Forward pass caches; backward pass multiplies local gradients.
6. **Loss choice:** MSE for regression, softmax + cross-entropy for classification (gradient $\hat{p} - y$ does not vanish).
7. **Gradient problems:** vanishing/exploding come from repeated multiplication with depth. Fix with He/Xavier init + ReLU + clipping + normalisation.
8. **Optimisers:** SGD $\rightarrow$ momentum $\rightarrow$ RMSProp $\rightarrow$ Adam. Adam is fast; SGD+momentum often generalises better.
9. **Generalisation:** bias-variance; L1 (sparse) vs L2 (smooth); dropout, BatchNorm, early stopping, augmentation.

10. **CNNs:** local connectivity + weight sharing + translation robustness. One filter = one feature map.
11. **Sequences:** RNN memory → vanishing gradient → LSTM (3 gates, additive cell state) → GRU (2 gates) → attention → Transformer.
12. **Evaluation:** accuracy is not enough — add calibration (ECE), robustness (sensitivity curves), uncertainty (entropy).
13. **Diagnostics:** gradients, activations, parameters. Learning rate is the usual root cause.
14. **Tuning:** LR first; random search beats grid; Bayesian optimisation is most sample-efficient; reproducibility must be engineered.
15. **Responsible AI:** interpretability vs explainability; LIME/SHAP/Grad-CAM; bias sources; fairness metrics conflict (impossibility result).